

Hardware-Optimized Quantized Graphs for Fast Similarity Search

Anonymous Author(s)

Abstract

Similarity search is a fundamental primitive in modern data systems, where approximate nearest neighbor search (ANNS) enables high recall with low query latency. Quantized graph (QG) methods achieve state-of-the-art ANNS performance by combining graph traversal with distance evaluation over compact vector codes. However, existing designs are limited by (i) inefficient distance evaluation stemming from high-precision lookup tables that restrict register-level parallelism; (ii) hardware-constrained quantization that prevents flexible bit allocation and high-precision distance computation; and (iii) traversal strategies that underutilize hardware parallelism or rely on redundant connectivity, leading to high memory overhead. We present *Optimized Quantized Graph* (OQG), a novel hardware-optimized QG method that achieves high-performance approximate similarity search through the co-design of flexible quantization, efficient and accurate distance evaluation, and a traversal strategy that exposes parallelism without increasing memory footprint. OQG decouples lookup precision from accumulation precision using compact low-bit lookup tables with high-precision register-level accumulation, preserving distance accuracy during traversal. OQG further enables flexible bit allocation through a novel multi-register design for high-bit quantized codes and introduces a new multi-path traversal strategy that aggregates candidates into large evaluation batches without increasing graph connectivity or memory overhead. We conduct the largest experimental evaluation of QG methods to date across 30 diverse datasets and show that OQG consistently outperforms the strongest existing methods in accuracy, throughput, and memory efficiency, with comparable indexing performance. At high recall (Recall@100 $\approx$ 0.98), OQG achieves up to 5x higher throughput compared to SymQG while reducing memory up to 1.8x. We release the OQG library as open source to ensure reproducibility, enable further research, and ease integration.

1 Introduction

Similarity search plays a central role in contemporary data infrastructures [2, 3, 56, 111, 135], enabling efficient retrieval over large vector collections for tasks such as retrieval-augmented generation [46, 77, 79] and machine learning [19, 26, 115, 121, 138, 142, 143]. *Exact nearest neighbor search* guarantees retrieval of the true nearest neighbors but incurs prohibitive computational cost at scale, especially for high-dimensional data. *Approximate nearest neighbor search* (ANNS) addresses this challenge by efficiently retrieving near-matching vectors without guaranteeing the exact nearest ones, achieving recall close to exact search with much lower latency.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00

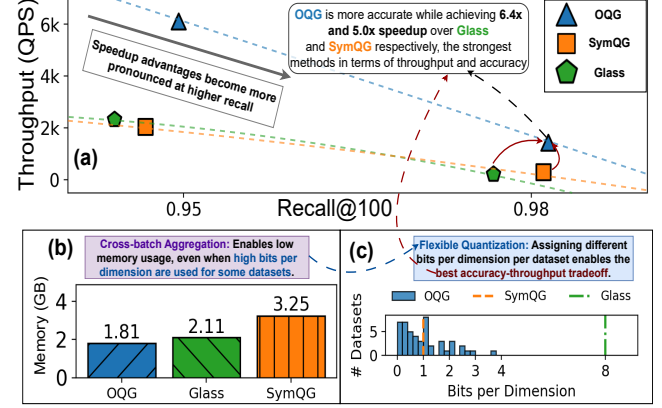


Figure 1: Average performance across 30 datasets shows that OQG outperforms the strongest ANNS methods (SymQG [53] and Glass [117]) in accuracy, throughput, and memory.

ANNS methods include graph [41, 89], quantization [48, 65], hashing [49, 60, 96], and tree [1, 42] methods. Among them, graph-based methods achieve state-of-the-art performance in recent benchmarks and studies [10, 31, 70, 80]. Graph methods construct a proximity graph that connects each vector to a small set of nearby neighbors and perform search via greedy, distance-guided traversal starting from an entry point. By exploiting local connectivity and short paths in the graph, graph methods efficiently narrow the search space and achieve high recall with low query latency.

Although graph methods are highly effective, distance computation on high-dimensional raw vectors remains expensive and often dominates query latency during traversal [148]. To reduce this cost, *quantized graph* (QG) methods [4, 53, 63, 117, 148] combine graph search with vector quantization, replacing exact distance computation with fast approximate evaluation on compact codes. Among quantization techniques, *Product Quantization* (PQ) is particularly attractive because PQ splits vectors into subspaces and represents each subspace by a learned centroid index, achieving high compression ratio and approximation accuracy [148]. Recent methods [6–8, 16] further accelerate PQ-based distance evaluation by exploiting CPU vectorization for parallel table lookup and distance accumulation, substantially reducing memory traffic and improving throughput. To further improve search throughput, some QG methods [63] build on these hardware-efficient PQ kernels and adopt a *redundancy connectivity* that stores neighbor codes alongside each node, enabling distance evaluation over an entire neighborhood via a single contiguous memory access during graph traversal.

Despite recent progress, three key properties remain critical for improving the performance of QG methods. Designs based on *Scalar Quantization* (SQ) [4, 117, 148] rely on a single shared codebook across all dimensions, limiting their ability to capture heterogeneous structures in high-dimensional data. While PQ-based approaches improve representational fidelity, existing hardware-accelerated designs [7, 8, 16, 62] restrict PQ to low bitwidths (e.g., 4 bits per subspace) to fit lookup tables into vector registers, allowing

Table 1: Comparison of high-performance ANNS. More ★ indicates better performance.

Methods	Key Properties			Evaluation Aspects		Performance		
	Low-bit Lookup with High-Precision Accumulation	Flexible Quantization Granularity	Memory-Efficient Hardware Acceleration	Rigorous Statistical Analysis	Number of Datasets	Accuracy-Throughput Trade-off	Memory Usage	Indexing Time
SymQG [53]	×	×	×	×	8	★★	★	★★★
Glass [117]	×	×	×	×	3	★★	★★★	★
HNSWLib [89]	×	×	×	×	11	★	★★★	★
OG-LVQ [4]	×	✓	×	×	10	★	★★★★	★
VSAQ [148]	×	×	×	×	8	★	★★★	★
NGT-QG [62]	×	✓	×	×	5	★	★	★
OQG (ours)	✓	✓	✓	✓	30	★★★★	★★★★	★★

only 16 centroids per subspace, significantly degrading quantization accuracy [8]. In addition, the precision of distance accumulation is critical for accurate traversal decisions. Low-precision accumulation quickly saturates when aggregating contributions from multiple subspaces, leading to inaccurate distance comparisons. For example, 8-bit accumulation provides only 255 distinct values, which must represent distances for millions of vectors; in practice, each value corresponds to thousands of candidates, making it difficult to distinguish them during traversal. As shown later (Figure 15), accumulated distances across subspaces often exceed 1000 distinct values on diverse datasets, far beyond the 8-bit range, indicating that saturation is a pervasive issue. However, achieving high-precision accumulation efficiently remains challenging. RaBitQ-based methods (e.g., SymQG) [45, 53] adopt QuickerADC-style [7, 8] distance evaluation but retain low-precision accumulation throughout traversal, limiting effective resolution. While extensions of QuickerADC support high-precision accumulation, they achieve this by using high-bit lookup tables, which increases lookup cost and reduces efficiency. As a result, existing approaches either rely on low-precision accumulation or incur high-cost lookup operations to maintain precision, lacking an efficient mechanism to achieve high-precision distance computation without increasing lookup cost.

Second, flexible quantization granularity remains limited, yet is essential for adapting to diverse dataset characteristics and accuracy targets. Existing QG methods [53, 117] adopt fixed per-dimension bitwidths (e.g., 8-bit in SQ or 1-bit in RaBitQ), restricting their ability to adjust encoding precision to different data distributions. While extensions such as Ex-RaBitQ [43] support multiple bits per dimension, combining them with QG significantly increases memory usage due to the need to store higher-precision codes for all graph nodes, making such designs impractical at scale. As shown in Figure 1(c), different datasets favor different bits per dimension to achieve the best accuracy-throughput trade-off. Fixed-bitwidth designs cannot capture such variations, leading to suboptimal performance across diverse datasets. Finally, memory-efficient hardware acceleration remains a key challenge for vector search. In practice, graph expansions often expose only a limited number of neighbors (e.g., 16), making it difficult to form large candidate batches and leaving hardware parallelism underutilized [53]. However, efficient execution requires processing larger batches (e.g., 32 or 64 candidates) to achieve high throughput. SymQG addresses this by introducing redundant connectivity to increase per-step candidate counts, but this comes at the cost of substantial memory overhead. Consequently, current approaches fail to effectively balance throughput and memory efficiency. In addition to the above methodological limitations, prior works evaluate performance only on a limited number of datasets and lack statistical validation of their findings, hindering generalization across diverse datasets.

To address the aforementioned limitations, we propose *Optimized Quantized Graph* (OQG), a hardware-accelerated QG method that jointly improves distance evaluation efficiency, quantization flexibility, and traversal-level hardware utilization. First, OQG introduces a novel distance evaluation pipeline that decouples lookup precision from accumulation precision. By employing compact low-bit lookup tables, OQG maximizes the number of distance values processed per register, while widening and accumulating distances in high precision only at the register level, preserving numerical resolution during traversal while significantly increasing evaluation throughput. Second, OQG extends this pipeline to support flexible bit allocation by orchestrating multiple registers to represent and process high-bit quantized codes, removing register-aligned bitwidth constraints and enabling accurate distance accumulation under heterogeneous quantization settings. Finally, OQG introduces a new traversal strategy that explores multiple search paths in parallel and aggregates their candidates into large evaluation batches, improving hardware utilization without increasing graph connectivity. In contrast to approaches that rely on expanded or redundant neighborhoods to expose parallelism, this design preserves sparse graph structure while maintaining high throughput and low memory overhead. Table 1 summarizes these properties and shows that OQG is the first QG method to simultaneously achieve flexible quantization, high-precision distance evaluation, and memory-efficient hardware-aware traversal.

To evaluate the effectiveness of OQG, we conduct the largest study of QG methods to date, spanning 30 diverse datasets and employing rigorous statistical analysis. Across all settings, OQG consistently dominates the accuracy-throughput-memory trade-off compared to the strongest high-performance QG methods [4, 53, 62, 117, 148]. Figure 1(a) shows that for high recall (e.g., Recall@100 $\approx$ 0.98), OQG achieves up to 6.4x and 5.0x higher throughput than Glass and SymQG, respectively, while reducing memory usage by up to 1.8x (Figure 1(b)). Experiments on 100-million-scale datasets further corroborate these findings, demonstrating that OQG maintains its accuracy and throughput advantages at scale while reducing memory usage by up to 32% and indexing time by up to 8.6x. We release the OQG library as open source to ensure reproducibility and enable integration into existing vector search systems [9].

We first review the relevant background and preliminaries in Section 2. We then summarize our contributions as follows:

- **Flexible bit allocation via multi-register.** We orchestrate multiple registers to support high-bit codes, removing register-aligned bitwidth constraints to enable flexible bit allocation (Section 3.1).
- **Low-bit lookup with high-precision accumulation.** We design a vectorized PQ lookup kernel that uses low-bit lookup tables with high-precision distance accumulation to enable efficient and accurate distance evaluation (Section 3.2).
- **Cross-batch candidates aggregation.** We aggregate candidate nodes from multiple search paths to form large batches for parallel distance evaluation without increasing graph connectivity, thereby reducing memory overhead (Section 3.3).
- **Comprehensive evaluation.** We evaluate OQG and the strongest QG baselines on 30 diverse datasets and two large-scale datasets, with rigorous statistical analysis, showing consistent improvements in throughput, accuracy, and memory (Sections 4 and 5). We discuss related work at Section 6 and conclude at Section 7.

2 Background and Preliminaries

This section presents the relevant preliminaries and related methods. We first introduce PQ for efficient ANNS (Section 2.1), then summarize graph-based ANNS methods and their traversal strategies (Section 2.2), and finally review quantized graph methods that integrate vector quantization into graph search (Section 2.3). Together, these methods outline the algorithmic foundations for searching billions of vectors within a second.

2.1 PQ for Efficient ANNS

High-dimensional ANNS requires evaluating many query–vector distances, making exact computation expensive at scale; a compact approximation that supports efficient evaluation is therefore essential. In this subsection, we introduce how PQ quantizes vectors and how it enables efficient distance approximation.

Training and Encoding. Let $\mathcal{X} = \{x_1, \dots, x_N\} \subset \mathbb{R}^D$ denote the database vectors. PQ splits each vector into M sub-vectors, $x = [x^{(1)}, \dots, x^{(M)}]$. For each subspace m , PQ applies k -means [87] clustering to the corresponding sub-vectors in the training data to learn a codebook $C_m = \{c_{m,1}, \dots, c_{m,K}\}$ with $K = 2^b$ centroids. PQ encodes a vector x by replacing each $x^{(m)}$ with the index of its nearest centroid, using the *Euclidean distance*:

$$\text{PQ}(x) = [k_1, \dots, k_M], \quad k_m = \arg \min_{j \in [K]} \|x^{(m)} - c_{m,j}\|.$$

Top- k Retrieval. Given a query q , PQ approximates distances using precomputed centroid distances stored in *lookup table (LUT)*. For each subspace m , PQ constructs a LUT

$$L_m[j] = \|q^{(m)} - c_{m,j}\|^2, \quad j \in [K],$$

where $c_{m,j}$ denotes the j -th centroid of the m -th codebook. The approximate distance between q and a database vector encoded as $(k_1, \dots, k_M)$ is computed as follows:

$$\hat{d}(q, x) = \sum_{m=1}^M L_m[k_m],$$

which requires only M table lookups and additions. To perform top- k retrieval, PQ evaluates $\hat{d}(q, x)$ for candidate vectors generated by the index and returns the k vectors with the smallest approximate distances. This lookup-and-accumulate formulation enables efficient distance evaluation with low memory traffic, making PQ well-suited for large-scale ANNS.

Hardware-Accelerated Distance Approximation. *Single Instruction, Multiple Data (SIMD)* enables a processor to apply the same operation to many data elements in parallel, making it well suited for workloads with regular, data-parallel structure. PQ distance evaluation naturally fits this model because it consists of simple, uniform operations—table lookups and additions—that can be applied to many candidate vectors simultaneously. However, storing full-precision LUT entries as 32-bit floating-point numbers makes it difficult to keep entire LUTs resident in SIMD registers; for example, a single 256-entry float LUT requires 1 KB, far exceeding the capacity of vector registers. As a result, each lookup incurs a memory access, which becomes a major performance bottleneck during ANNS.

To eliminate these memory accesses, recent systems *quantize* the LUT values from 32-bit floats to compact low-bit integers so that LUTs can fit entirely inside SIMD registers and be processed using

wide integer vector instructions. Formally, for a floating-point LUT $L_m[j]$, a quantized LUT is produced by an affine mapping

$$\tilde{L}_m[j] = \text{clip}(\alpha_m L_m[j] + \beta_m), \quad \tilde{L}_m[j] \in \{0, \dots, 2^b - 1\},$$

where b is the bitwidth. Distance is evaluated as follows:

$$\tilde{d}(q, x) = \sum_{m=1}^M \tilde{L}_m[k_m].$$

Using 8-bit integers is a common compromise: the range $[0, 255]$ preserves enough resolution for approximate distances while allowing many LUT entries to be packed into SIMD registers.

Leveraging this idea, existing work [7, 8, 16, 32, 57] restricts PQ to 4-bit sub-quantizers so that each LUT has only 16 entries, allowing the entire table to fit in registers and enabling very fast in-register lookups. The drawback is reduced accuracy due to the small codebooks. PQFS [6] improves accuracy by extending in-register computation to 8-bit (256-entry) codebooks: each LUT is divided into multiple 16-entry segments, and the correct segment is chosen based on the PQ code before SIMD accumulation. While segmentation preserves accuracy better than 4-bit PQ, it introduces additional control-flow overhead, reduces runtime flexibility, and supports only up to eight subspaces. These limitations motivate supporting 8-bit-per-subspace PQ distance evaluation, but *without* subspace restrictions or segmentation overhead in OQG.

2.2 Accelerating ANNS with Graph Structures

Graph-based ANNS indices link each vector to a small set of neighbors and perform greedy, distance-guided traversal. NSW [17, 72] is an early design that achieves strong accuracy by enriching local links with a few long-range connections, enabling search to quickly jump toward promising regions. However, its single-layer structure can require many greedy steps on large datasets. HNSW [89] addresses this by building a hierarchy of layers: upper layers use sparse long-range links for coarse navigation, while lower layers provide dense local refinement. Three parameters govern its behavior: R , the maximum number of neighbors per node; $EFConstruction$, the candidate list size used during insertion; and $EFSearch$, the number of candidate nodes retained during querying. Increasing any of these parameters gives the algorithm access to more candidate neighbors, which improves the chances of finding closer nodes and thus increases accuracy. However, doing so also raises construction cost or query latency because more distance evaluations and more bookkeeping must be performed. This hierarchical design underlies many state-of-the-art ANNS methods [53, 117].

Building on the same greedy, distance-guided traversal principle, NSG [41] constructs a sparse graph with neighbors that are both close in distance and well distributed in direction. This design preserves connectivity and enables effective navigation during search. Its performance is primarily controlled by the maximum node degree and the search candidate size: increasing these parameters improves recall by exposing more candidate neighbors, but increases memory usage and query cost due to additional distance evaluations. These graph structures form the foundation for efficient quantized graph designs, which we introduce next to further reduce distance computation cost while preserving strong search performance.

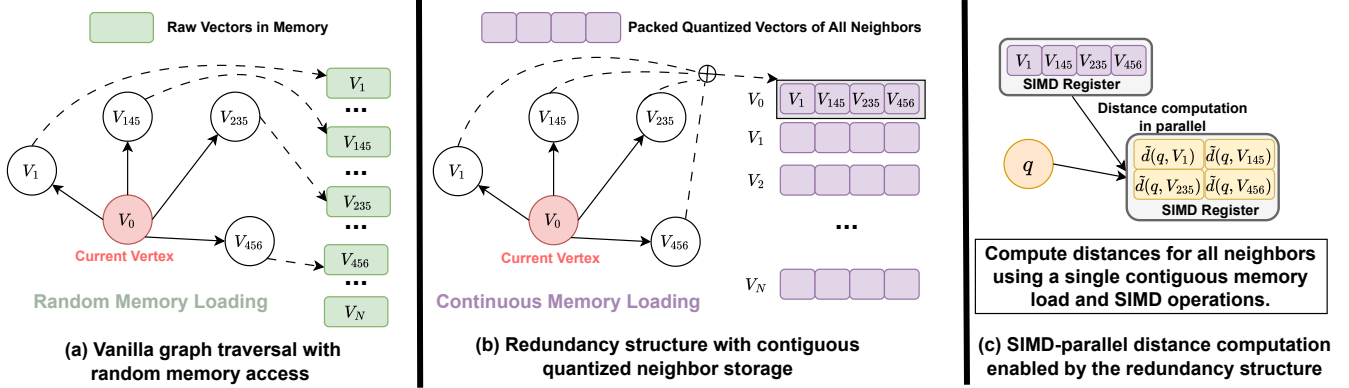


Figure 2: Redundancy structure enabling SIMD-efficient quantized graph traversal: (a) random memory access in vanilla traversal; (b) contiguous storage of quantized neighbors using the redundancy structure; (c) SIMD-parallel distance computation.

2.3 Quantized Graphs: Enabling Billion-Scale Search within a Second

Quantization has emerged as a powerful technique for accelerating graph-based ANNS by reducing the cost of distance approximation during neighbor expansions, enabling search over billions of vectors within a second [10, 70]. Instead of computing distances in full precision, graph search can operate on compact codes produced by *scalar quantization* (SQ) [149], achieving substantially lower computation and memory traffic. SQ maps each dimension to a few discrete levels, and integrating it into graph structures has been shown to be highly effective; for example, Glass [117] incorporates SQ into graph traversal and achieves state-of-the-art throughput in public benchmarks [10, 70]. VSAG [148] further accelerates search by dynamically tuning memory prefetch parameters to reduce latency.

Beyond SQ-based designs, several methods incorporate more expressive quantization into graph traversal. OG-LVQ [4] extends Vamana [64] by replacing floating-point vectors with *local vector quantization* (LVQ) codes, reducing both memory and distance computation cost. NGT-QG [63] applies PQ within the ONNG structure to accelerate traversal, while SymQG [53] replaces PQ with RabbitQ [45], an error-bounded quantizer that improves quantization accuracy while maintaining efficient distance computation. As illustrated in Figure 2, to further reduce traversal overhead, these methods commonly adopt a *redundancy structure*—storing quantized representations of all neighbors directly within each node—which enables contiguous memory access and efficient batch distance evaluation at the cost of additional memory.

Although these methods enable fast and accurate graph search and show strong results on certain benchmarks, their performance is not consistently robust across diverse datasets and often incurs substantial memory overhead. This motivated OQG, which achieves strong and stable performance across 30 diverse datasets while reducing memory overhead, as we demonstrate later (Section 5.1.1).

3 Optimized Quantized Graph

We present the core design of OQG, focusing on hardware-conscious distance evaluation and traversal in quantized graphs. We first introduce a novel flexible 8-bit PQ-based distance approximation pipeline that enables fully register-resident SIMD lookups (Section 3.1). We

then extend this design with 16-bit distance accumulation to preserve distance resolution and support accurate graph navigation (Section 3.2). Finally, we propose a new connectivity augmentation, which aligns graph traversal with wide-vector SIMD execution by exposing sufficiently large neighbor batches per expansion step (Section 3.3). These components enable high-throughput, accurate navigation, and a low memory footprint in OQG.

3.1 Flexible PQ-based Distance Approximation

This subsection begins by motivating the need for flexible quantization, and then presents our SIMD-efficient PQ distance evaluation design, featuring a novel multi-register organization that enables efficient high-bit lookup with flexible bit allocation, and lays the foundation for high-precision distance accumulation.

Motivation. Existing QG methods such as NGT-QG rely on fixed 4-bit sub-quantizers, limiting *each subspace* to 16 centroids and introducing substantial distance distortion [8], a limitation that cannot be resolved by simply increasing the total bit budget. Unlike SQ and RaBitQ, which rely on fixed bitwidth *per dimension*, OQG enables flexible bit allocation to accommodate heterogeneous quantization requirements, achieving better throughput–accuracy trade-offs across datasets, as illustrated later (e.g., Figure 9). To realize this flexibility, OQG leverages 8-bit codes—the minimum addressable storage unit on modern CPUs—as a building block, while its novel multi-register design allows large LUTs and multiple subspace-specific LUTs to be held simultaneously in SIMD registers. This removes the register-aligned bitwidth constraint in prior SIMD-based methods and enables flexible, higher-bit quantization without sacrificing SIMD efficiency. Although QuickerADC supports 8-bit quantization, its implementation is typically constrained to fixed subspace configurations (e.g., 8 or 16) [8] and lacks a foundation for efficiently supporting high-precision distance accumulation, which we address in Section 3.2. In contrast, our design enables true flexible subspace configurations while achieving up to 1.75x faster kernel-level distance evaluation under the same subspaces (Table 3).

Quantized Lookup Tables. We begin by quantizing the floating-point lookup table $L_m[j]$, whose 256 entries stored as 32-bit floating-point values require 1 KB—small enough to reside in the L1 cache (e.g., 32 KB) [6] but too large to fit into a small number of vector registers (e.g., 64 B each). Following prior LUT-quantization

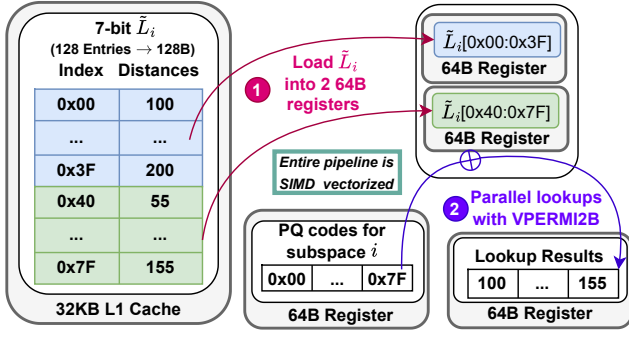


Figure 3: Lookup with 7-bit indices. The 128-entry distance table is loaded into two 64-byte SIMD registers, and 7-bit PQ codes (0x00–0x7F) are mapped to their corresponding distances via parallel VPERMI2B lookups.

work [6–8, 16], we map each $L_m[j]$ to an 8-bit integer and obtain a compact table $\tilde{L}_m[j]$. This reduces the table to 256 bytes. With 8-bit entries, a 512-bit vector register holds 64 consecutive table values, so storing all 256 entries of $\tilde{L}_m$ requires exactly four registers arranged as four contiguous 64-entry blocks. We preload these four registers once before scanning and keep them live throughout the search, enabling fully register-resident lookups and avoiding repeated table loads from the L1 cache. As a result, each SIMD lookup produces distances for a full batch of 64 PQ codes in parallel, fully utilizing the available vector width.

Lookup with 7-bit indices. To achieve flexible and fully register-resident lookup under wide SIMD execution, we organize the distance table into register-sized blocks and design a lookup scheme that scales with bitwidth. We first present a 7-bit lookup formulation that naturally fits within two SIMD registers and enables efficient 64-lane parallel lookup. We then extend this design to support full 8-bit PQ codes without sacrificing SIMD efficiency. Figure 3 illustrates our SIMD lookup procedure for 7-bit PQ indices. As shown in Step ①, we load the 128-entry distance table $\tilde{L}_m$ into two 64-byte SIMD registers, each holding 64 consecutive entries. We then process a vector of 7-bit PQ codes that serve as indices into this 128-entry table. As shown in Step ②, we apply a two-source byte permutation instruction (VPERMI2B), which treats the two registers as a single logical table and performs parallel lookups for all lanes, producing one distance value per lane entirely in registers. Using a single permute instruction, we thus obtain 64 lookup results in parallel. Because $\tilde{L}_m$ remains resident in registers across iterations, the entire pipeline is SIMD-vectorized and incurs no additional L1 table accesses. Unlike prior 8-bit PQ implementations that rely on fixed layouts, our novel design maintains full register residency while enabling a flexible number of subspaces.

Extending to Full 8-bit Lookup with Flexible Encoding. Figure 4 illustrates how we extend the 7-bit SIMD lookup to support full 8-bit PQ indices. As shown in Step ①, we first load the 256-entry distance table $\tilde{L}_m$ into registers, organized as two 128-entry halves (lower and upper), with each half spanning two 64-byte SIMD registers. In Step ②, each 8-bit PQ code is split into a 7-bit index and a 1-bit selector. A mask-test instruction such as VPTESTMB extracts the most significant bit of each code to form a lane-wise selector mask, while a bitwise AND with 0x7F clears the high bit to produce a vector of 7-bit indices. As shown in Step ③, we reuse

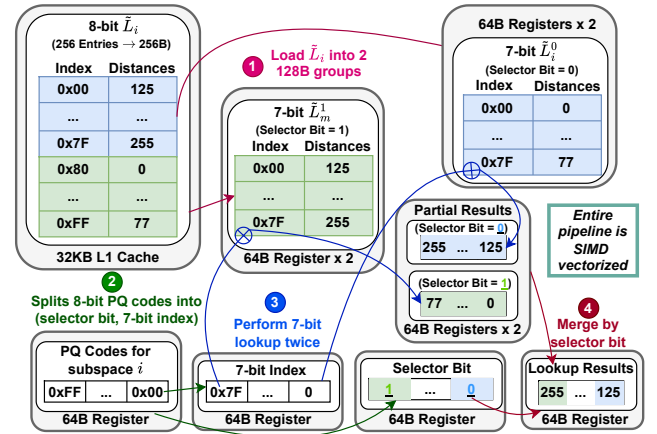


Figure 4: Full 8-bit lookup via two 7-bit SIMD lookups. PQ codes are split, both sub-LUTs are queried in parallel, and results are merged using the selector bit.

the 7-bit lookup mechanism by applying two VPERMI2B permutes in parallel, querying the lower and upper halves of $\tilde{L}_m$ with the same 7-bit index vector. Finally, in Step ④, a masked byte blend (VPBLENDMB) uses the selector mask to choose, lane by lane, between the two partial lookup results. This partition-and-select design extends the 7-bit SIMD lookup to the full 8-bit domain while keeping all entries of $\tilde{L}_m$ resident in registers and the entire pipeline fully SIMD-vectorized. More importantly, by organizing LUTs across multiple SIMD registers, our design naturally supports flexible encoding, allowing an arbitrary number of subspaces to be processed and enabling variable bit allocation across dimensions. This represents a substantial departure from prior designs, enabling true flexibility in OQG for the first time.

Distance Accumulation. We accumulate the per-subspace lookup outputs using the unsigned add instruction VPADDUSB, performing lane-wise accumulation over a SIMD batch of 64 distance values in parallel. After processing all subspaces, the vector of 64 aggregated distances remains in registers. With quantized tables $\tilde{L}_m$ stored in registers, table access performed by VPERMI2B, segment choice by VPBLENDMB, and accumulation by VPADDUSB, the kernel evaluates 8-bit PQ distances for a batch of 64 candidates without any LUT traffic to L1 during the main scanning loop. However, accumulating distances in 8-bit precision constrains the dynamic range and can lead to saturation as the number of subspaces increases; we address this limitation in the next subsection.

3.2 High-Precision Distance Accumulation

This subsection first motivates the need for high-precision (i.e., 16-bit) accumulation by analyzing its impact on navigation accuracy, and then presents our SIMD-friendly design that enables high-resolution distance accumulation while preserving 8-bit lookup.

Motivation. Accurate graph traversal critically depends on preserving fine-grained distance ordering. However, as shown in Figure 5, using 8-bit accumulation leads to significantly lower navigation accuracy compared to 16-bit accumulation across datasets. This gap highlights that low-precision accumulation fundamentally limits traversal quality. The root cause lies in the mismatch between the distance scale and the representable range. As shown in Figure 15(a), the average navigation distances after LUT quantization typically

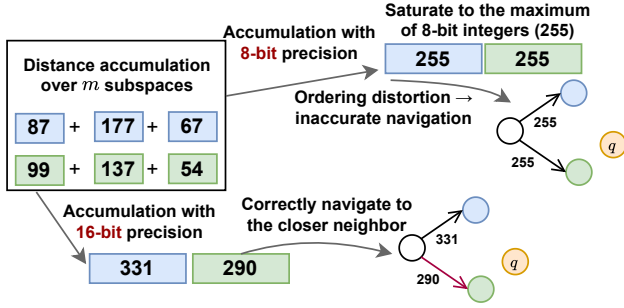


Figure 5: Impact of accumulation precision on graph navigation. Accumulating distances with 8-bit precision causes saturation at the maximum value (255), leading to ordering distortion and incorrect traversal decisions. Using 16-bit precision preserves relative distance ordering, enabling accurate navigation toward the closer neighbor.

fall in the range of 10^3 to 10^4 . These values far exceed the 8-bit accumulation range of $[0, 255]$, but remain well within the capacity of 16-bit precision. As a result, 8-bit accumulation saturates during navigation, degrading neighbor ordering and leading to unreliable traversal decisions. More fundamentally, 8-bit accumulation provides only 256 distinct bins to differentiate distances among millions of vectors, forcing many candidates to share identical scores and eliminating fine-grained distinctions. A straightforward solution is to use 16-bit precision. Indeed, prior work such as QuickerADC adopts 16-bit LUTs together with 16-bit accumulation to avoid saturation. However, this approach significantly increases memory traffic and lookup cost, as each LUT entry doubles in size and reduces SIMD efficiency. In contrast, we observe that high-precision accumulation does not require high-precision lookup. By retaining compact 8-bit LUTs while elevating only the accumulator to 16-bit precision, we eliminate saturation without sacrificing SIMD efficiency. A 16-bit range of 0–65,535 allows all subspaces to fully exploit the 8-bit LUT dynamic range (0–255) while avoiding overflow as long as: $255m \leq 65,535 \iff m \leq 257$. Since practical PQ configurations use far fewer subspaces (e.g., 8–64), 16-bit accumulation preserves full-resolution LUT values and supports accurate distance estimation for reliable graph navigation. This novel decoupled design avoids the overhead of 16-bit LUTs while eliminating saturation, and achieves up to 1.75x speedup over the 16-bit LUT-based design of QuickerADC at the kernel level (Table 3).

Widening 8-bit Lookup Distances for 16-bit Accumulation. Building on the 8-bit lookup mechanism introduced in Section 3.1, we next describe how per-subspace distances are accumulated without loss of resolution. Each lookup produces a 64-lane vector of 8-bit distances in the range $[0, 255]$. Accumulating these values directly in 8-bit precision would quickly overflow, forcing each subspace to occupy only a small fraction of the available dynamic range. To preserve full resolution, we therefore widen each 8-bit distance to 16-bit precision before any accumulation takes place.

As illustrated in Figure 6, distance widening and accumulation proceed in two stages. First, each 8-bit PQ lookup yields a batch of 64 distances, matching the natural width of modern wide-vector SIMD instructions and enabling fully parallel processing (Step ①). We then split this batch into two contiguous blocks of 32 distances (Step ②) and widen each block from unsigned 8-bit to unsigned

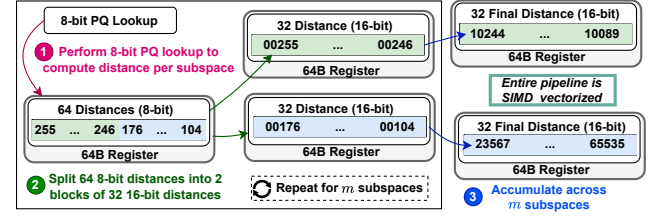


Figure 6: 16-bit distance accumulation via SIMD widening. 8-bit PQ distances are split into two 32-lane blocks, widened to 16-bit precision, and accumulated across subspaces.

16-bit using VPMOVBW, which preserves the numerical value of every distance. In the second stage, we accumulate the widened distances across subspaces using standard 16-bit addition (VPADDW) (Step ③). Two 16-bit accumulators are maintained in registers—one per 32-lane block—and updated as the scan iterates over all m subspaces. The entire pipeline remains SIMD-vectorized while safely accommodating the full 0–255 contribution from each subspace. Algorithm 1 summarizes this procedure.

By widening each 8-bit distance before summation, the lookup path (VPERM12B, VPTESTMB, VPAND, VPBLENDMB) continues to produce accurate per-subspace distances, while the 16-bit accumulation path retains these values without overflow or loss of precision. After all subspaces are processed, the two 32-lane accumulators are written back as 64 16-bit PQ distances. This design enables OQG to fully exploit 8-bit sub-quantizers while maintaining high-resolution accumulated distances for efficient graph traversal.

3.3 Cross-Batch Candidates Aggregation

As discussed in Section 3.2, our PQ kernel can evaluate a batch of 64 approximate distances in parallel using SIMD instructions. To fully utilize this capability, prior QG methods such as SymQG [53] expose a large number of candidate neighbors per expansion step by relying on a single node. Specifically, they increase the effective graph degree to match the SIMD width (e.g., 32/64 neighbors) and materialize the quantized representations of all neighbors in a contiguous memory layout. As illustrated in Figure 7(a), this design enables a single SIMD computation over a large batch of neighbors from one node.

Motivation. While effective for hardware utilization, this design tightly couples the SIMD batch size with graph connectivity. Achieving full SIMD utilization requires increasing the node degree to B , which introduces significant redundancy in memory. Concretely, each node must store the quantized codes of all B neighbors, resulting in a memory overhead of BN (e.g., $64N$). Moreover, enforcing high-degree connectivity alters the graph structure, increasing construction complexity and potentially introducing redundant local neighbors that contribute limited new information during search. This design, therefore, trades memory efficiency and structural sparsity for SIMD utilization, making it less suitable for large-scale or memory-constrained settings. This limitation motivates our novel design that decouples SIMD utilization from graph connectivity.

Cross-Batch Candidates Aggregation with Multi-Path Traversal. To address this limitation, we decouple the SIMD batch size from the per-node graph degree by advancing multiple search paths in parallel and aggregating candidates across them. Instead of requiring a single node to expose B neighbors, we maintain multiple active nodes from different paths, each contributing a small

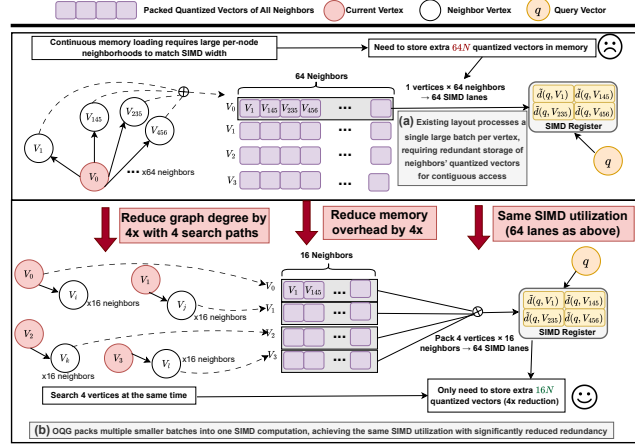


Figure 7: Advancing multiple search paths in parallel and aggregating their neighbor sets enables full SIMD utilization without requiring high per-node degree.

neighbor set, and combine these candidates into a SIMD-aligned batch. Formally, let B denote the SIMD batch size (e.g., $B = 64$), and let each active node expose d neighbors with $d \ll B$. Rather than relying on one high-degree node, we jointly process $e = \lfloor B/d \rfloor$ active nodes and aggregate their neighbor sets to form a batch of size B . For example, when $B = 64$ and $d = 16$, we process $e = 4$ paths in parallel and combine the neighbors of their current frontier nodes, yielding a full SIMD batch ($4 \times 16 = 64$), as illustrated in Figure 7(b). This design preserves SIMD efficiency without requiring high-degree connectivity at each node. Because each path only expands a small neighborhood, the graph can remain compact, reducing memory overhead from BN to dN (e.g., $64N \rightarrow 16N$). At the same time, aggregating candidates across paths provides enough parallelism to fully utilize SIMD registers while retaining contiguous memory access within each node expansion.

Multi-Entry Selection from Intermediate Layers. To enable effective aggregation, we must identify multiple entry points that are both close to the query and sufficiently diverse. Selecting entry points from the top layer ($L_{\max}$) yields candidates that are too far from the query, while selecting from the bottom layer (L_0) leads to highly redundant candidates due to dense local connectivity. We therefore select entry points from an intermediate layer (e.g., L_1), which provides a natural balance between proximity and diversity. Nodes at this level are already close to the query due to hierarchical traversal, while still maintaining structural diversity across the graph. We select the top- k candidates from this layer and propagate them to the bottom layer, where their neighborhoods are aggregated for SIMD evaluation.

By combining multi-entry selection with cross-batch candidates aggregation, the resulting SIMD batches cover diverse regions of the search space while remaining focused around the query. In this sense, multi-entry selection determines *which* nodes contribute to each batch, while cross-batch candidates aggregation determines *how* their distances are evaluated efficiently with reduced memory.

4 Experimental Settings

In this section, we describe the experimental settings used to evaluate OQG and the state-of-the-art quantized graph methods.

Algorithm 1: 16-bit distance accumulation for 8-bit PQ

Input: Query subspaces $\{q^{(m)}\}_{m=1}^M$; PQ codes for a 64-vector batch $\{k_m[0..63]\}_{m=1}^M$

Output: Approximate distances $\hat{d}_{16}[0..63]$ (16-bit, exact accumulation)

Parameters;

$b \leftarrow 8; K \leftarrow 2^b$; // $K = 256$ centroids per subspace

Precompute and quantize lookup tables;

```

for  $m \leftarrow 1$  to  $M$  do
  for  $j \leftarrow 0$  to  $K - 1$  do
     $L_m[j] \leftarrow \|q^{(m)} - c_{m,j}\|^2$ ;
   $\tilde{L}_m \leftarrow \text{QUANTIZEToU8}(L_m)$ ;
   $R0 \leftarrow \text{LOADU8x64}(\tilde{L}_m[0..63])$ ,  $R1 \leftarrow \text{LOADU8x64}(\tilde{L}_m[64..127])$ ;
   $R2 \leftarrow \text{LOADU8x64}(\tilde{L}_m[128..191])$ ,  $R3 \leftarrow \text{LOADU8x64}(\tilde{L}_m[192..255])$ ;

```

Scan a 64-vector batch with 16-bit accumulation;

```

accLo16  $\leftarrow \text{SETZEROU16x32}()$ ;
accHi16  $\leftarrow \text{SETZEROU16x32}()$ ;
for  $m \leftarrow 1$  to  $M$  do
  codes  $\leftarrow \text{LOADU8x64}(k_m[0..63])$ ;
  sel  $\leftarrow \text{VPTSTMB}(\text{codes})$ , idx  $\leftarrow \text{VPANDD}(\text{codes}, 0x7F)$ ;
  lo8  $\leftarrow \text{VPERMI2B}(R0, R1, \text{idx})$ , hi8  $\leftarrow \text{VPERMI2B}(R2, R3, \text{idx})$ ;
  dist8  $\leftarrow \text{VPBLENDB}(sel, hi8, lo8)$ ;
  dLo16  $\leftarrow \text{VPMOVZXBW}(\text{Lower32B}(\text{dist8}))$ ;
  dHi16  $\leftarrow \text{VPMOVZXBW}(\text{Upper32B}(\text{dist8}))$ ;
  accLo16  $\leftarrow \text{VPADDW}(\text{accLo16}, dLo16)$ ;
  accHi16  $\leftarrow \text{VPADDW}(\text{accHi16}, dHi16)$ ;
 $\hat{d}_{16}[0..31] \leftarrow \text{STOREU16x32}(\text{accLo16})$ ;
 $\hat{d}_{16}[32..63] \leftarrow \text{STOREU16x32}(\text{accHi16})$ ;
return  $\hat{d}_{16}$ ;

```

Datasets. As shown in Table 2, our evaluation spans 30 diverse datasets, and 2 large-scale datasets with 100 million vectors each. While prior work typically evaluates on a limited set of our benchmark datasets [4, 53, 117, 148], we substantially expand the evaluation to a broader range of domains, including image, text, audio, astronomy, and seismology, in order to capture more diverse workloads. To characterize this increased diversity, we quantify dataset difficulty using *local intrinsic dimensionality (LID)* [14, 123], where a larger LID indicates a more challenging search space.

Environment. We conduct all experiments on an Ubuntu 24.04 server with two Intel(R) Xeon(R) Platinum 8380@2.30 GHz CPUs and 256 GB of RAM. We enable *simultaneous multithreading (SMT)* and thread pinning for all methods. We train all methods using 128 threads and execute all query evaluations on a single thread. For the two large-scale datasets, we use a server with the same CPU model and software environment but equipped with 512 GB of RAM.

Implementation. We implement OQG using hand-optimized SIMD kernels in C++ with seamless support for AVX-512. We compile OQG with flags `-Ofast -fno-alias -fno-alias -funroll-loops`. We provide a header-only C++ library together with Python bindings, enabling users to train their own datasets and achieve billion-scale query throughput with only a few lines of Python code. To ensure full reproducibility, we also release all parameter settings used for every evaluated dataset as part of our library distribution [9].

Evaluation Metrics. We evaluate accuracy using the standard Recall@R metric, following established benchmarks and prior work [10, 53, 70]. For a query q , let $G_R(q)$ denote its top- R ground-truth neighbors, and let $A_R(q)$ denote the top- R results returned by a method. Recall@R is defined as: $\text{Recall@R} = \frac{1}{|Q|} \sum_{q \in Q} \frac{|A_R(q) \cap G_R(q)|}{R}$, which measures the fraction of true top- R neighbors recovered among

Table 2: Datasets statistics, where D denotes the vector dimension, N_X the number of database vectors, LID the local intrinsic dimensionality, and N_M the dataset size on memory.

Dataset	D	N_X	LID	N_M	Dataset	D	N_X	LID	N_M
Common Benchmark Datasets ($N_X \leq 5$ million)									
ARXIV [70]	768	1000000	60.9	2.86GB	ASTRO1M [102]	256	1000000	69.1	980MB
AUDIO [138]	192	53387	18.6	39MB	BIGANN [35]	128	1000000	30.3	492MB
CCNEWS [70]	768	495328	53.1	1.42GB	CIFAR [80]	512	50000	41.8	98MB
CRAWL [80]	300	1989995	56.0	2.23GB	DEEP1M [80]	256	1000000	39.4	980MB
GIST [93]	960	1000000	66.5	3.58GB	GLOVE [100]	100	1192514	46.6	459MB
IMAGENET [80]	150	2340373	30.9	1.32GB	IQUIQUE [126]	256	578853	27.8	567MB
LANDMARK [70]	768	760757	29.7	2.18GB	LLAMA [70]	128	256921	43.5	126MB
MILLIONSONG [80]	420	992272	31.6	1.56GB	MNIST [80]	784	69000	25.2	207MB
NEIC [126]	256	1000000	15.0	980MB	NETFLIX [86]	300	17770	12.5	20MB
NOTRE [80]	128	332668	25.7	164MB	NUSWIDE [80]	500	268643	82.1	513MB
SALD1M [122]	128	1000000	19.8	492MB	SEISMIC1M [61]	256	1000000	42.4	980MB
SIFT [85]	128	1000000	31.2	492MB	SPACEV1M [25]	100	1000000	57.8	385MB
SUN [80]	512	79106	43.4	155MB	TEXTTOIMAGE [35]	200	1000000	35.0	767MB
TINY5M [59]	384	5000000	60.9	7.17GB	UKBENCH [80]	128	1097907	24.8	540MB
UQV [80]	256	1000000	19.0	980MB	YAHOO [70]	384	677305	52.2	995MB
Large Scale Datasets ($N_X = 100$ million)									
SIFT100M [85]	128	100000000	29.2	49GB	DEEP100M [13]	96	100000000	32.1	37GB

the returned R candidates, averaged over all queries. To quantify throughput, we report *queries per second (QPS)*, i.e., the number of queries processed per second.

Statistical Analysis. We employ a ranking-based statistical evaluation to compare accuracy across all datasets. For each dataset, methods are ranked according to Recall@100, and their average rank summarizes overall accuracy performance. Statistical significance is assessed at the 95% confidence level using the Friedman test [39], followed by the Nemenyi test [92] to determine whether rank differences exceed a critical threshold. Results are summarized using a *Critical Difference (CD)* diagram, which places methods along a horizontal axis according to their average ranks; methods connected by a horizontal bar do not differ significantly, whereas the absence of a connection indicates a statistically significant accuracy difference. To complement this global analysis, we report *pairwise recall comparison plots* that compare methods on each dataset. For each pairwise comparison, we apply the Wilcoxon signed-rank test [125] and report the corresponding p-value (p_w), which indicates whether the observed recall differences across datasets are statistically significant. At the 95% confidence level, a difference is considered statistically significant when $p_w \leq 0.05$.

Evaluated Methods and Parameter Settings. We compare our method against six high-performance ANNS libraries, including five that support quantized graph search—SymQG [53], Glass (Zilliz) [117], NGT (Yahoo Japan) [62], OG-LVQ (Intel) [4], and VSAG (Ant Group) [148]—as well as the vanilla HNSWlib [89]. Because different quantized graph methods expose different quantization parameters, we adopt a *parameter search* strategy to ensure a fair comparison. For each method and dataset, parameter search enumerates all supported quantization configurations and selects the one that achieves the highest throughput while maintaining a recall of at least 0.95. If a method cannot reach 0.95 recall on a dataset, we instead use a recall threshold of 0.9. For all baselines except NGT, we fix *EFConstruction* to 600 and vary $R \in \{32, 64\}$, following prior work [53] (see Section 2.2 for details). NGT exposes a large number of training parameters; therefore, we follow the default settings recommended in its official library. During evaluation, we vary the search-time parameters to obtain accuracy-throughput trade-off curves. We apply this *parameter search* strategy as follows: (i) OQG builds on product quantization and requires selecting the number of subspaces; we perform a grid search over the number

of subspaces to identify the optimal setting. For OQG, we fix *EFConstruction* to 600 and vary $R \in \{8, 16\}$. Unlike prior quantized graph methods such as SymQG, which require the graph degree to be a multiple of the SIMD batch size (e.g., 32) to fully utilize vector lanes, OQG does not impose such constraints. This flexibility is enabled by our cross-batch aggregation design, which decouples graph connectivity from SIMD execution while maintaining high hardware efficiency. (ii) SymQG integrates RabbitQ [45] with graph search. RabbitQ quantizes each dimension to a single bit, eliminating the need for parameter tuning. We do not consider the graph-based Ex-RabbitQ [43], as its support for multi-bit quantization per dimension leads to proportional memory overhead, which is impractical at scale, and it currently lacks an official implementation. (iii) Glass supports scalar quantization using either 8-bit unsigned integers or 8-bit floating-point values; we evaluate both options and select the better-performing configuration for each dataset. We adopt NSG as the base graph, since it consistently achieves better search performance in terms of throughput and accuracy than HNSW within the Glass library in our experiments. (iv) OG-LVQ combines LVQ [4] with the VAMANA graph [64] and provides multiple quantization variants, including LVQ8x8, LVQ4x8, and LVQ8. We evaluate all supported variants, as well as the unquantized setting, and select the best configuration per dataset. (v) NGT provides graph structures such as ONNG together with PQ-based quantization but exposes a large number of quantization parameters. In our parameter search, we evaluate both the quantized configuration (following the official recommendations for 4-bit PQ per subspace) and the unquantized configuration, and select the better-performing setting for each dataset. We include the unquantized option because NGT only supports 4-bit PQ, which can substantially reduce accuracy on some datasets. (vi) VSAG supports scalar quantization with 16-bit or 8-bit precision; we evaluate both quantization precisions and select the optimal setting for each dataset.

5 Experimental Results

In this section, we evaluate OQG in terms of search performance, robustness, and scalability. We report recall-throughput-memory results on 30 diverse datasets, including an analysis by dataset difficulty using LID (Section 5.1.1). We then conduct rigorous statistical analysis to validate the robustness of OQG across diverse datasets (Section 5.1.2). After that, we examine indexing time (Section 5.1.3), followed by scalability experiments on 100-million-scale datasets (Section 5.1.4). We further conduct ablation studies (Section 5.1.5), followed by an analysis of graph degree and its impact on search performance (Section 5.1.6), and kernel-level latency comparisons with QuickerADC (Section 5.1.7). Finally, we study the navigation accuracy under different distance precision settings (Section 5.2), and the sensitivity of OQG to different top- k settings to explain performance trends across retrieval regimes (Section 5.3).

5.1 Performance Analysis

5.1.1 Search Performance Analysis. We evaluate search performance using recall-throughput curves together with memory usage. We first report the average results over all 30 datasets, which span diverse domains including text, image, audio, embedding vectors, and astronomy. We note that, due to the inclusion of 30 datasets, the recall of several methods differs from prior studies where all methods achieved high performance. As shown in Figure 8(a), OQG

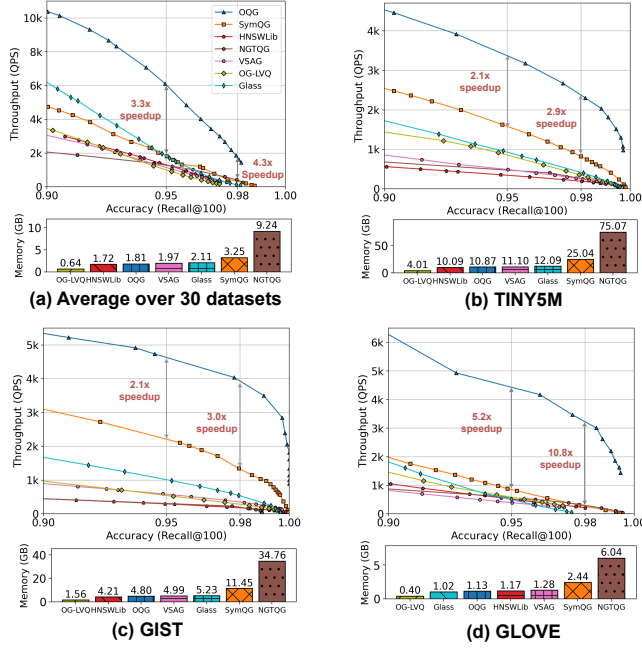


Figure 8: Search performance comparison of OQG against baselines, averaged across 30 datasets, and on three representative datasets (TINY5M, GIST, and GLOVE).

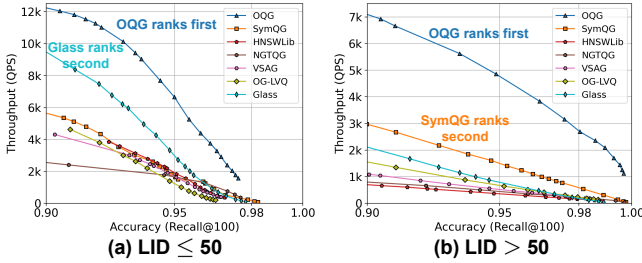


Figure 9: Search performance under different LID regimes. We group datasets into (a) LID ≤ 50 and (b) LID > 50 settings, and report recall-throughput curves for all methods, illustrating the robustness of OQG across dataset difficulty.

consistently achieves the best overall accuracy-throughput trade-off and remains on the Pareto frontier across the full recall range. In contrast, existing quantized graph baselines do not consistently outperform vanilla HNSWlib and often suffer throughput degradation in medium- and high-recall regimes. Compared with the strongest QG baseline (SymQG and Glass), OQG delivers 3.3x and 4.3x higher throughput at Recall@100 = 0.95 and 0.98, respectively. Importantly, these gains are achieved without increased memory overhead: OQG uses nearly the same average memory as HNSWlib (1.81 GB vs. 1.72 GB), while remaining substantially more memory-efficient than SymQG (3.25 GB) and Glass (2.11 GB). Although SymQG improves over vanilla HNSWlib in some recall ranges, the redundant neighborhood structure it maintains to keep SIMD lanes occupied introduces much higher memory overhead, whereas OQG achieves higher accuracy and throughput with HNSWlib-like memory through the efficient cross-batch candidates aggregation. We further examine three representative datasets, TINY5M, and GLOVE, which are also used in prior work [4, 53, 148]. As shown

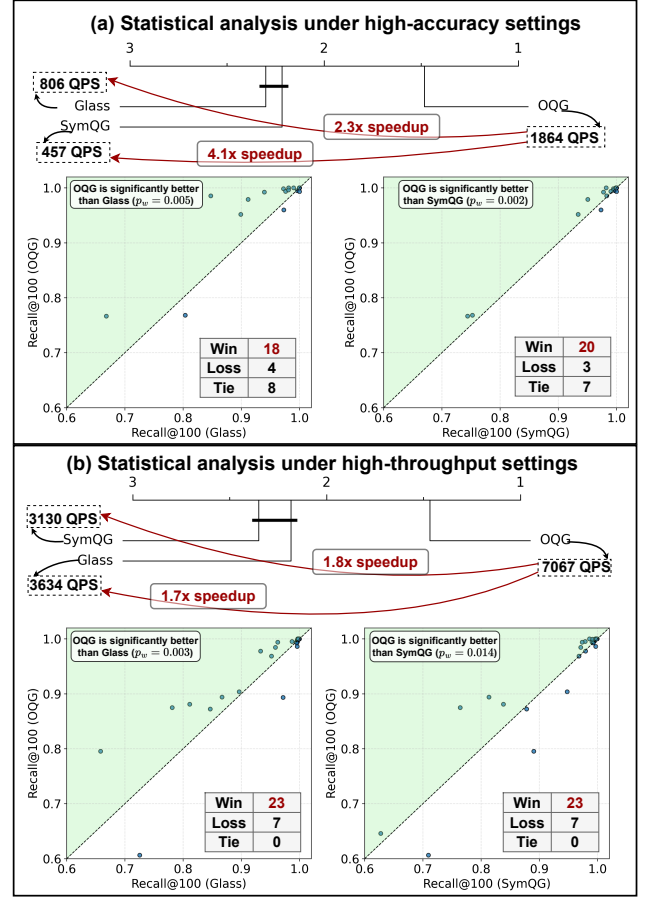


Figure 10: Statistical Analysis of OQG against 2 strongest baselines (Glass and SymQG) on 30 diverse datasets.

in Figure 8(b)–(d), OQG consistently dominates the baselines over nearly the entire recall range. Compared with SymQG, it achieves 2.1x–5.2x at Recall@100 = 0.95, and 3.0x–10.8x at Recall@100 = 0.98, with even larger gains in the high-recall regime, while using memory substantially less than SymQG. These results confirm that OQG delivers the best overall search performance by jointly improving accuracy, throughput, and memory efficiency across diverse datasets.

To further understand why some quantized graph methods fail to consistently outperform the vanilla HNSWlib, we group datasets by LID. We classify datasets with LID ≤ 50 as *easy* and those with LID > 50 as *hard*. As shown in Figure 9, Glass achieves higher throughput than HNSWlib on easy datasets, whereas SymQG exhibits noticeable performance degradation. In contrast, on hard datasets, Glass fails to maintain its advantage, while SymQG consistently delivers higher throughput. These results indicate that existing quantized graph methods are sensitive to dataset difficulty and lack robustness across heterogeneous workloads. By combining higher-bit PQ with an augmented graph structure, OQG consistently achieves the highest throughput across both easy and hard datasets, demonstrating strong robustness to dataset difficulty.

5.1.2 Statistical Analysis. While the average performance results demonstrate that OQG achieves superior accuracy, throughput,

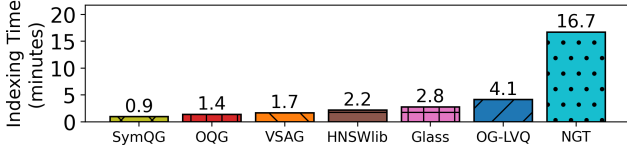


Figure 11: Average indexing time measured across 30 datasets.

and memory efficiency, they do not capture the statistical significance of these improvements across datasets. To rigorously assess whether these gains are consistent across datasets, we conduct a statistical analysis. Figure 10(a) presents the statistical analysis under a high-accuracy operating point, where all methods are configured such that their Recall@100, averaged across 30 datasets, is at least 0.95. We first assess accuracy differences using the CD diagram, which orders methods along a horizontal axis by their average Recall@100 rank, with methods further to the right indicating better accuracy; a connecting bar denotes no statistically significant difference at the 95% confidence level (see Section 4 for details). Under this high-accuracy operating point, OQG attains the best average rank with no CD overlap against either Glass or SymQG, indicating a statistically significant accuracy advantage at the 95% confidence level, while *simultaneously* achieving 2.3x–4.1x throughput speedups under the same configuration. This conclusion is further supported by *pairwise recall comparisons* using the Wilcoxon signed-rank test, which confirms statistical significance at a 95% confidence level ($p_w \leq 0.05$) for this operating regime. Across datasets in this setting, OQG attains higher recall than Glass on 18 datasets and than SymQG on 20 datasets. Figure 10(b) reports results under a throughput-oriented operating point, where all methods operate with averaged Recall@100 $\in [0.90, 0.95]$. Even in this regime, OQG achieves around 1.8x speedups while maintaining statistically significantly higher recall than both Glass and SymQG at a 95% confidence level. All pairwise recall comparisons using the Wilcoxon signed-rank test indicate statistically significant improvements in favor of OQG, with higher recall on 30 datasets compared to Glass and on 28 datasets compared to SymQG.

5.1.3 Indexing Time. Figure 11 reports the average indexing time across all 30 diverse datasets. OQG requires only 1.4 minutes on average to build the index, making it comparable to SymQG and faster than all other baselines. This result shows that OQG delivers substantial query-time gains while keeping indexing overhead low.

5.1.4 Scalable Experiments. We evaluate scalability on two large-scale datasets, SIFT100M and DEEP100M, each containing 100 million vectors. As shown in Figure 12, OQG consistently achieves the best accuracy–throughput trade-off across the full recall range on both datasets, consistently dominating all baselines. At Recall@100 = 0.95, OQG delivers 1.8x–2.0x higher throughput than the strongest baseline, with similar gains observed across other recall levels. Importantly, these improvements are achieved while using substantially less memory than both SymQG and Glass. For example, on SIFT100M, OQG uses 82.62 GB, compared to 131.14 GB and 121.93 GB for SymQG and Glass, respectively, corresponding to a 1.5x–1.6x reduction. A similar trend is observed on DEEP100M, where OQG reduces memory usage by 1.3x–1.5x compared to SymQG and Glass, demonstrating that OQG simultaneously improves accuracy, throughput, and memory efficiency at scale.

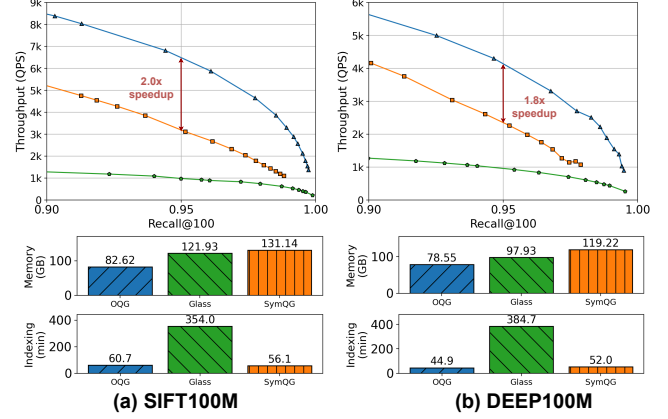


Figure 12: OQG vs. strongest baselines (SymQG, Glass) on two large-scale datasets for search performance.

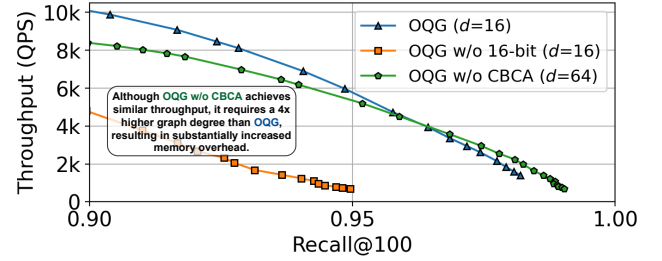


Figure 13: We compare the complete OQG with variants that disable 16-bit distance accumulation (OQG w/o 16-bit) and cross-batch candidates aggregation (OQG w/o CBCA), reporting results averaged over 30 diverse datasets.

In addition, OQG builds the index in 60.7 and 44.9 minutes on SIFT100M and DEEP100M, respectively, remaining comparable to SymQG and significantly faster than Glass (by 5.8x–8.6x). Therefore, OQG scales effectively to large-scale datasets, achieving superior search performance with practical memory and indexing costs.

5.1.5 Ablation Study. We conduct an ablation study to quantify the individual impact of 16-bit distance accumulation and cross-batch candidates aggregation (CBCA) on search performance. Figure 13 compares the complete OQG against two variants: disabling 16-bit accumulation (OQG w/o 16-bit) and disabling CBCA (OQG w/o CBCA). Results are averaged over 30 diverse datasets and reported as throughput vs. accuracy. We analyze these two ablations separately to isolate how they contribute to OQG’s overall performance. **Disabling 16-bit accumulation** leads to a substantial throughput drop at high recall levels. While OQG w/o 16-bit performs comparably at lower recall, its curve degrades rapidly beyond Recall@100 ≈ 0.9 . This behavior highlights the importance of higher-precision accumulation: accumulating per-subspace distances in 8-bit precision compresses the effective distance range, weakens distance discrimination during traversal, and forces additional graph expansions to reach high recall, reducing overall throughput.

Disabling CBCA changes the throughput–accuracy trade-off in a different way. As shown in Figure 13, OQG w/o CBCA only becomes competitive with the full OQG in the high-recall regime, and even slightly exceeds it at some operating points. However, this variant requires a much larger degree ($d = 64$ rather than $d = 16$), relying on

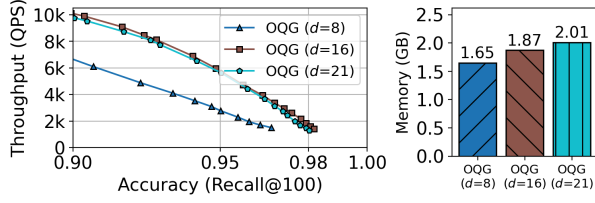


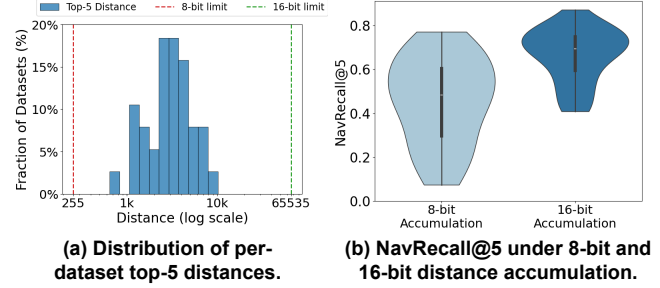
Figure 14: Effect of degree on the search performance in OQG.
Table 3: Kernel-level latency of distance evaluation (per vector) for OQG and QuickerADC.

#Subspaces	QuickerADC (ns)	OQG (ns)	Speedup
8	0.612	0.350	1.75x
16	1.199	0.755	1.59x

redundant supplementary structures to expose enough candidates for batched distance evaluation. In contrast, the full OQG achieves comparable or better performance over most of the recall range with a much smaller degree, showing that CBCA is effective in preserving throughput while substantially reducing memory overhead.

5.1.6 Achieving High Throughput without Large Degree. We analyze how OQG achieves high throughput without relying on large graph degrees. In OQG, the degree d determines the number of entry nodes e ($e = \lfloor 64/d \rfloor$), which controls how many candidates can be exposed for batched distance evaluation. Figure 14 compares OQG under three settings ($d = 8, 16, 21$), chosen by fixing the number of entry nodes $e \in \{8, 4, 3\}$ and setting $d = \lfloor 64/e \rfloor$, yielding $d = 8$, $d = 16$, and $d = 21$, respectively. Increasing d improves the accuracy-throughput trade-off, as more entry nodes expose larger candidate sets per expansion. Compared to $d = 8$, both $d = 16$ and $d = 21$ achieve substantially higher throughput across the recall range, with the largest gains at medium to high recall. However, the benefit quickly saturates: $d = 21$ provides no improvements over $d = 16$, especially in the high-recall regime. Importantly, this result shows that OQG does not require large degrees to achieve high throughput. With cross-batch aggregation, $d = 16$ already exposes sufficient candidates to sustain efficient batched distance evaluation, avoiding the need for redundant connectivity. In contrast, increasing d further yields diminishing returns while increasing memory overhead. As shown in Figure 14(right), memory grows from 1.65 GB ($d = 8$) to 1.87 GB ($d = 16$) and 2.01 GB ($d = 21$). Overall, $d = 16$ provides the best trade-off, achieving near-optimal throughput with moderate memory usage.

5.1.7 Kernel-Level Latency of Distance Evaluation. We compare the kernel-level latency of distance evaluation between OQG and QuickerADC under different numbers of subspaces on a random generated dataset of 1 million vectors. This experiment isolates the distance computation kernel without graph traversal to evaluate raw SIMD efficiency. Table 3 reports the average per-vector latency and the corresponding speedup. Across both settings, OQG consistently outperforms QuickerADC, achieving 1.59x–1.75x speedup. This improvement stems from a more efficient SIMD execution pattern: OQG directly operates on 8-bit codes with compact lookup tables, enabling full utilization of 64-lane vector registers and reducing instruction overhead. Overall, OQG achieves fast kernel-level distance evaluation, enabling improved end-to-end ANNS performance.



(a) Distribution of per-dataset top-5 distances.
(b) NavRecall@5 under 8-bit and 16-bit distance accumulation.
Figure 15: Analysis of navigation distance scale and navigation accuracy across 30 diverse datasets.

5.2 Graph Navigation Analyses

5.2.1 16-bit Distance Accumulation for Accurate Navigation. As discussed in Section 3.2, 8-bit accumulation severely limits the dynamic range of PQ distance aggregation across subspaces, causing distance saturation and degraded neighbor ordering during graph traversal. Our analysis over 30 diverse datasets shows that traversal distances commonly exceed the 8-bit range but fit within 16-bit precision, motivating 16-bit accumulation for reliable navigation.

Navigation Distance and Scale Analysis. We mimic a single navigation step as follows. Given a query vector q and a graph node v with neighbor set $\mathcal{N}(v)$ (with $|\mathcal{N}(v)| = 64$), we compute distances $\{d(q, u) \mid u \in \mathcal{N}(v)\}$. We then record the *top-5 distance*, defined as the 5-th smallest value, $d^{(5)}(q, v)$. For each dataset, we average $d^{(5)}(q, v)$ over all visited nodes v and all queries q , yielding a single representative navigation distance per dataset. Figure 15(a) plots the distribution of these per-dataset Top-5 distances across all 30 datasets (log scale). The dashed vertical lines indicate the maximum representable values under 8-bit (255) and 16-bit (65,535) distance accumulation. As shown in Figure 15(a), the per-dataset Top-5 navigation distances consistently fall above the 8-bit accumulation limit (255) and below the 16-bit limit (65,535) across all 30 datasets. This means 8-bit accumulation saturates during navigation, collapsing meaningful distance differences among neighbors and degrading traversal decisions, whereas 16-bit accumulation preserves the required dynamic range and enables reliable graph navigation.

Navigation Accuracy Analysis. To directly quantify the impact of accumulation precision on navigation decisions, we measure *NavRecall@5*. For each query q and node v , we compute exact distances to all neighbors using full-precision (32-bit floating-point) arithmetic and treat the top-5 nearest neighbors as ground truth, denoted by $\mathcal{N}_5^*(q, v)$. We then compute predicted neighbor rankings using PQ distances accumulated with either 8-bit or 16-bit precision, yielding $\hat{\mathcal{N}}_5(q, v)$. Navigation accuracy is defined as: $\text{NavRecall@5} = |\mathcal{N}_5^*(q, v) \cap \hat{\mathcal{N}}_5(q, v)|/5$, and is averaged over all nodes and queries. Figure 15(b) reports the distribution of *NavRecall@5* across datasets under 8-bit and 16-bit distance accumulation. As shown in Figure 15(b), 8-bit accumulation yields substantially lower and more variable *NavRecall@5*, reflecting frequent mis-ordering among neighbors caused by distance saturation. In contrast, 16-bit accumulation consistently achieves higher *NavRecall@5* with reduced variance, closely matching the exact-distance ordering. Together, these results show that graph navigation operates in a high-precision regime, making 16-bit distance accumulation essential for reliable traversal and robust performance.

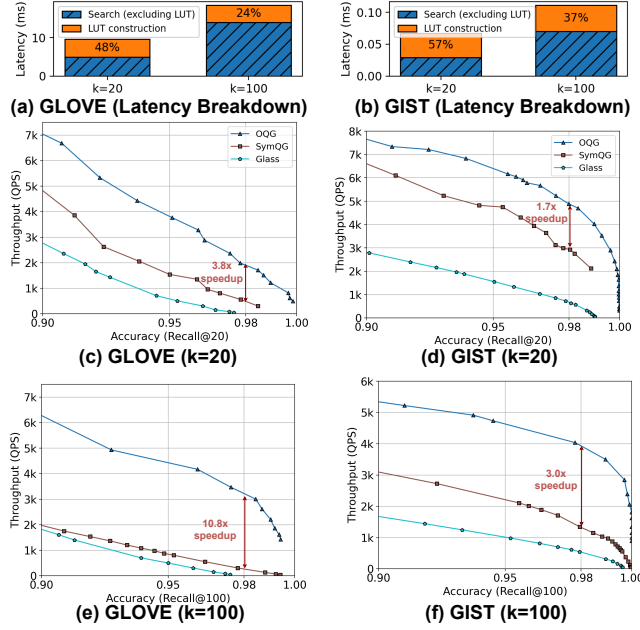


Figure 16: We evaluate OQG against the two strongest baselines (SymQG and Glass) on GLOVE and GIST for $k \in \{20, 100\}$. Panels (c)-(f) report throughput-recall curves obtained by varying $EFSearch$, while panels (a)-(b) fix $EFSearch = k$ and show the search latency breakdowns.

5.3 Understanding OQG across top- k Regimes

The top- k parameter affects both the search workload and the impact of distance computation and traversal optimizations in graph-based ANNS. Smaller k values favor low-latency navigation with limited candidate expansion, whereas larger k values require evaluating more candidates and incur heavier distance computation. To study this effect, we evaluate OQG across different top- k regimes and compare it against the two strongest baselines, SymQG and Glass. Figures 16(c)-(f) show throughput-recall curves on GLOVE and GIST for $k \in \{20, 100\}$. OQG consistently outperforms both baselines, achieving up to 3.8x (GLOVE) and 1.7x (GIST) speedups at $k = 20$, and substantially larger gains of 10.8x and 3.0x at $k = 100$. These results indicate that OQG benefits increasingly in computation-intensive regimes, where its SIMD-optimized distance evaluation amortizes overhead over a larger number of candidates.

To analyze cost composition across different k values, Figures 16(a) and (b) report latency breakdowns with $EFSearch = k$. At small $k = 20$, LUT construction accounts for a large portion of total latency (48% on GLOVE and 57% on GIST). As k increases, this fixed overhead is amortized: at $k = 100$, LUT construction contributes only 24% of total latency on GLOVE and 37% on GIST. Consequently, performance at larger k is dominated by SIMD-efficient distance computation rather than setup costs, explaining the substantially larger speedups observed in high- k regimes. Overall, OQG delivers consistent gains across all top- k settings, with its advantages becoming most pronounced as computation intensity increases.

6 Related Work

This section complements the background in Section 2 and positions our approach within the broader ANNS literature.

Graph methods for ANNS [4, 23, 24, 32, 33, 40, 41, 53, 62–64, 67, 69, 88, 89, 99, 110, 117, 128, 129, 133, 134, 136, 140, 148] organize vectors into proximity graphs that enable efficient neighbor-based traversal during search. In parallel, quantization techniques [4, 11, 12, 38, 43, 45, 48, 52, 54, 57, 65, 78, 84, 90, 94, 106, 114, 120, 144, 149] compress high-dimensional vectors into compact discrete codes, enabling fast approximate distance evaluation with reduced memory footprint. More recent studies bridge these two directions by integrating quantization compression directly into graph nodes [4, 33, 53, 62, 117, 148], forming *quantized graph* indices that accelerating distance computation during graph traversal. Beyond graph and quantization approaches, ANNS has also been studied through hashing and tree paradigms. Hash methods [49, 58, 60, 76, 86, 105, 108, 124, 124, 146, 147] project high-dimensional vectors into lower-dimensional representations using multiple hash functions, such that nearby points are likely to share the same hash codes; query processing retrieves candidates from matching hash buckets and refines results via exact distance computation. Tree methods [1, 15, 18, 21, 42, 91, 118] partition the data space into hierarchical regions, with each leaf node containing a disjoint subset of points. During search, the query traverses the tree from the root by selecting child nodes that are closest to the query, and candidate points stored in the reached leaves are then exhaustively evaluated. Many tree methods rely on hyperplane partitioning [1, 20, 22, 29, 34, 73, 83, 97, 98, 101, 104, 113, 116, 119, 131, 150], which uses randomly oriented or axis-aligned hyperplanes to construct balanced trees and guide queries toward regions likely to contain nearest neighbors.

Beyond indexing techniques, filtered ANNS [28] incorporates attribute-based constraints into similarity search. Equality-based methods [50, 81, 95, 112, 127, 137] target discrete attributes by maintaining separate indices or inverted structures for each filter value, whereas range-filter methods [36, 66, 82, 99, 130, 141] handle continuous attributes through multi-dimensional partitioning, quantization, and metadata-aware pruning. In parallel, substantial effort has focused on accelerating ANNS across hardware platforms. On CPUs, prior work reduces computation via progressive distance estimation [30, 44, 75, 132], or accelerates low-bit distance accumulation using vectorized operations [6–8, 16, 74]. GPU-based methods [27, 47, 55, 69, 71, 107, 128, 145] leverage massive parallelism to speed up both traversal and distance computation, while FPGA designs [5, 67, 68, 103, 139] emphasize pipelined execution and on-chip data reuse to improve energy efficiency. Moreover, several methods [37, 51, 109] extend existing ANNS indices to support streaming, cloud-based, and federated search services.

7 Conclusion

We presented OQG, a hardware-accelerated QG method that addresses key limitations in existing QG search systems. By jointly enabling flexible quantization, high-precision distance accumulation, and cross-batch candidates aggregation, OQG improves accuracy, throughput, and memory across a wide range of operating regimes. Extensive experiments on 30 diverse datasets, including two large-scale datasets, demonstrate that OQG consistently outperforms state-of-the-art quantized graph baselines under both accuracy-critical and throughput-oriented settings. These results show that careful co-design of quantization, distance computation, and graph traversal is essential for high-performance ANNS.

References

- [1] [n.d.]. annoy. <https://github.com/spotify/annoy>
- [2] [n.d.]. mongo atlas vector search. <https://www.mongodb.com/products/platform/atlas-vector-search>
- [3] [n.d.]. tencent cloud vector database. <https://www.tencentcloud.com/products/vdb>
- [4] Cecilia Aguerrebere, Ishwar Bhati, Mark Hildebrand, Mariano Tepper, and Ted Willke. 2023. Similarity search in the blink of an eye with compressed indices. *arXiv preprint arXiv:2304.04759* (2023).
- [5] Murat Alçın, İhsan Pehlivan, and İsmail Koyuncu. 2016. Hardware design and implementation of a novel ANN-based chaotic generator in FPGA. *Optik* 127, 13 (2016), 5500–5505.
- [6] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2016. Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. In *42nd International Conference on Very Large Data Bases*, Vol. 9. 12.
- [7] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2017. Accelerated nearest neighbor search with quick adc. In *Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval*. 159–166.
- [8] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2019. Quicker adc: Unlocking the hidden potential of product quantization with simd. *IEEE transactions on pattern analysis and machine intelligence* 43, 5 (2019), 1666–1677.
- [9] Anonymous. 2026. OQG Library. https://anonymous.4open.science/r/OQG_Anonymous-DB5B/README.md
- [10] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems* 87 (2020), 101374. <https://doi.org/10.1016/j.is.2019.02.006>
- [11] Artem Babenko and Victor Lempitsky. 2014. Additive quantization for extreme vector compression. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 931–938.
- [12] Artem Babenko and Victor Lempitsky. 2015. Tree quantization for large-scale similarity search and classification. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 4240–4248.
- [13] Artem Babenko and Victor Lempitsky. 2016. Efficient indexing of billion-scale datasets of deep descriptors. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2055–2063.
- [14] Jonathan Bac, Evgeny M Mirkes, Alexander N Gorban, Ivan Tyukin, and Andrei Zinoviyev. 2021. Scikit-dimension: a python package for intrinsic dimension estimation. *Entropy* 23, 10 (2021), 1368.
- [15] Alina Beygelzimer, Sham Kakade, and John Langford. 2006. Cover trees for nearest neighbor. In *Proceedings of the 23rd international conference on Machine learning*. 97–104.
- [16] David W Blalock and John V Guttag. 2017. Bolt: Accelerated data mining with fast vector compression. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 727–735.
- [17] Marian Boguna, Dmitri Krioukov, and Kimberly C Claffy. 2009. Navigability of complex networks. *Nature Physics* 5, 1 (2009), 74–80.
- [18] Leonid Boytsov and Bileg Saikhhan Naidan. 2013. Learning to prune in metric and non-metric spaces. *Advances in Neural Information Processing Systems* 26 (2013).
- [19] Peter Brusilovskii, Alfred Kobsa, and Wolfgang Nejdl. 2007. *The adaptive web: methods and strategies of web personalization*. Vol. 4321. Springer Science & Business Media.
- [20] Alessandro Camerra, Jin Shieh, Themis Palpanas, Thanawin Rakthanmanon, and Eamonn Keogh. 2014. Beyond one billion time series: indexing and mining very large time series collections with SAX2+. *Knowledge and information systems* 39, 1 (2014), 123–151.
- [21] Lawrence Cayton. 2008. Fast nearest neighbor retrieval for bregman divergences. In *Proceedings of the 25th international conference on Machine learning*. 112–119.
- [22] Manos Chatzakakis, Panagioti Fatourou, Eleftherios Kosmas, Themis Palpanas, and Botao Peng. 2023. Odyssey: A journey in the land of distributed data series similarity search. *arXiv preprint arXiv:2301.11049* (2023).
- [23] Manos Chatzakakis, Yannis Papakonstantinou, and Themis Palpanas. 2025. DARTH: Declarative Recall Through Early Termination for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–26.
- [24] Patrick Chen, Wei-Cheng Chang, Jyun-Yu Jiang, Hsiang-Fu Yu, Inderjit Dhillon, and Cho-Jui Hsieh. 2023. Finger: Fast inference for graph-based approximate nearest neighbor search. In *Proceedings of the ACM Web Conference 2023*. 3225–3235.
- [25] Qi Chen, Haidong Wang, Mingqin Li, Gang Ren, Scarlett Li, Jeffery Zhu, Jason Li, Chuanjie Liu, Lintao Zhang, and Jingdong Wang. 2018. SPTAG: A library for fast approximate nearest neighbor search.
- [26] Ting Chen, Lala Li, and Yizhou Sun. 2020. Differentiable product quantization for end-to-end embedding compression. In *International Conference on Machine Learning*. PMLR, 1617–1626.
- [27] Wei Chen, Jincai Chen, Fuhao Zou, Yuan-Fang Li, Ping Lu, and Wei Zhao. 2019. Robustiq: A robust ann search method for billion-scale similarity search on gpus. In *Proceedings of the 2019 on international conference on multimedia retrieval*. 132–140.
- [28] Yannis Chronis, Helena Caminal, Yannis Papakonstantinou, Fatma Özcan, and Anastasia Ailamaki. 2025. Filtered Vector Search: State-of-the-Art and Research Opportunities. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5488–5492.
- [29] Sanjoy Dasgupta and Yoav Freund. 2008. Random projection trees and low dimensional manifolds. In *Proceedings of the fortieth annual ACM symposium on Theory of computing*. 537–546.
- [30] Liwei Deng, Penghao Chen, Ximu Zeng, Tianfu Wang, Yan Zhao, and Kai Zheng. 2024. Efficient Data-aware Distance Comparison Operations for High-Dimensional Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2411.17229* (2024).
- [31] Magdalen Dobson, Zheqi Shen, Guy E Blelloch, Laxman Dhulipala, Yan Gu, Harsha Vardhan Simhadri, and Yihan Sun. 2023. Scaling graph-based anns algorithms to billion-size datasets: A comparative analysis. *arXiv preprint arXiv:2305.04359* (2023).
- [32] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). _eprint: 2401.08281.
- [33] Matthijs Douze, Alexandre Sablayrolles, and Hervé Jégou. 2018. Link and code: Fast indexing with graphs and compact regression codes. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 3646–3654.
- [34] Karima Echihabi, Panagioti Fatourou, Kostas Zoumpatianos, Themis Palpanas, and Houda Benbrahim. 2022. Hercules against data series similarity search. *arXiv preprint arXiv:2212.13297* (2022).
- [35] Karima Echihabi, Kostas Zoumpatianos, Themis Palpanas, and Houda Benbrahim. 2019. Return of the Lernaean Hydra: Experimental Evaluation of Data Series Approximate Similarity Search. *Proc. VLDB Endow.* 13, 3 (Nov. 2019), 403–420. <https://doi.org/10.14778/3368289.3368303>
- [36] Navid Eslami and Niv Dayan. 2024. Memento Filter: A Fast, Dynamic, and Robust Range Filter. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–27.
- [37] Zeheng Fan, Yuxiang Zeng, Zhuanglin Zheng, and Yongxin Tong. 2025. Fed-VSE: A Privacy-Preserving and Efficient Vector Search Engine for Federated Databases. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5371–5374.
- [38] Hakan Ferhatosmanoglu, Ertem Tuncel, Divyakant Agrawal, and Amr El Abbadi. 2000. Vector approximation based indexing for non-uniform high dimensional data sets. In *Proceedings of the ninth international conference on Information and knowledge management*. 202–209.
- [39] Milton Friedman. 1937. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the american statistical association* 32, 200 (1937), 675–701.
- [40] Cong Fu and Deng Cai. 2016. Efanna: An extremely fast approximate nearest neighbor search algorithm based on knn graph. *arXiv preprint arXiv:1609.07228* (2016).
- [41] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graphs. *PVLDB* 12, 5 (2019), 461 – 474. <https://doi.org/10.14778/3303753.3303754>
- [42] Keinosuke Fukunaga and Patrenahalli M. Narendra. 1975. A branch and bound algorithm for computing k-nearest neighbors. *IEEE transactions on computers* 100, 7 (1975), 750–753.
- [43] Jianyang Gao, Yutong Gou, Yuexuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. 2025. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.
- [44] Jianyang Gao and Cheng Long. 2023. High-dimensional approximate nearest neighbor search: with reliable and efficient distance comparison operations. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–27.
- [45] Jianyang Gao and Cheng Long. 2024. RaBitQ: quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.
- [46] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* 2, 1 (2023).
- [47] Vincent Garcia, Eric Debreuve, and Michel Barlaud. 2008. Fast k nearest neighbor search using GPU. In *2008 IEEE computer society conference on computer vision and pattern recognition workshops*. IEEE, 1–6.
- [48] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized product quantization. *IEEE transactions on pattern analysis and machine intelligence* 36, 4 (2013), 744–755.
- [49] Aristides Gionis, Piotr Indyk, Rajeev Motwani, and others. 1999. Similarity search in high dimensions via hashing. In *Vldb*, Vol. 99. 518–529.
- [50] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, and others. 2023. Filtered-diskann: Graph

- algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*. 3406–3416.
- [51] Shenghao Gong, Haobo Sun, Ziquan Fang, Liu Liu, Lu Chen, and Yunjun Gao. 2025. VStream: A Distributed Streaming Vector Search System. *Proceedings of the VLDB Endowment* 18, 6 (2025), 1593–1606.
- [52] Yunchao Gong, Svetlana Lazebnik, Albert Gordo, and Florent Perronnin. 2012. Iterative quantization: A procrustean approach to learning binary codes for large-scale image retrieval. *IEEE transactions on pattern analysis and machine intelligence* 35, 12 (2012), 2916–2929.
- [53] Yutong Gou, Jianyang Gao, Yuxuan Xu, and Cheng Long. 2025. SymphonyQG: Towards Symphonious Integration of Quantization and Graph for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–26.
- [54] Robert Gray. 1984. Vector quantization. *IEEE Assp Magazine* 1, 2 (1984), 4–29.
- [55] Fabian Groh, Lukas Ruppert, Patrick Wieschollek, and Hendrik PA Lensch. 2022. Ggnn: Graph-based gpu nearest neighbor search. *IEEE Transactions on Big Data* 9, 1 (2022), 267–279.
- [56] Rentong Guo, Xiaofan Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jigao Luo, Qianya Cheng, Weizhi Xu, Jiarui Luo, Frank Liu, and others. 2022. Manu: a cloud native vector database management system. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3548–3561.
- [57] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *International Conference on Machine Learning*. <https://arxiv.org/abs/1908.10396>
- [58] Qiang Huang, Jianlin Feng, Yikai Zhang, Qiong Fang, and Wilfred Ng. 2015. Query-aware locality-sensitive hashing for approximate nearest neighbor search. *Proceedings of the VLDB Endowment* 9, 1 (2015), 1–12.
- [59] Qiang Huang, Yifan Lei, and Anthony KH Tung. 2021. Point-to-Hyperplane Nearest Neighbor Search Beyond the Unit Hypersphere. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. 777–789.
- [60] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing*. 604–613.
- [61] I. R. I. for Seismology with Artificial Intelligence. 2018. Seismic Data Access. <http://ds.iris.edu/data/access/>
- [62] Masajiro Iwasaki. 2015. Neighborhood Graph and Tree for Indexing High-dimensional Data. *Yahoo Japan Corporation*. Retrieved August 22 (2015), 2020.
- [63] Masajiro Iwasaki and Daisuke Miyazaki. 2018. Optimization of indexing based on k-nearest neighbor graph for proximity search in high-dimensional data. *arXiv preprint arXiv:1810.07355* (2018).
- [64] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in neural information processing systems* 32 (2019).
- [65] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2010), 117–128.
- [66] Mengxu Jiang, Zhi Yang, Fangyuan Zhang, Guan hao Hou, Jieming Shi, Wenchao Zhou, Feifei Li, and Sibow Wang. 2025. DIGRA: A Dynamic Graph Indexing for Approximate Nearest Neighbor Search with Range Filter. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.
- [67] Wenqi Jiang, Hang Hu, Torsten Hoefer, and Gustavo Alonso. 2025. Fast graph vector search via hardware acceleration and delayed-synchronization traversal. *Proceedings of the VLDB Endowment* 18, 11 (2025), 3797–3811.
- [68] Wenqi Jiang, Marco Zeller, Roger Waleffe, Torsten Hoefer, and Gustavo Alonso. 2023. Chameleon: a heterogeneous and disaggregated accelerator system for retrieval-augmented language models. *arXiv preprint arXiv:2310.09949* (2023).
- [69] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [70] Elias Jääsaari, Ville Hyvönen, Matteo Ceccarelli, Teemu Roos, and Martin Aumüller. 2025. VIBE: Vector Index Benchmark for Embeddings. <https://arxiv.org/abs/2505.17810> eprint: 2505.17810.
- [71] Saim Khan, Somesh Singh, Harsha Vardhan Simhadri, Jyothi Vedurada, and others. 2024. Bang: Billion-scale approximate nearest neighbor search using a single gpu. *arXiv preprint arXiv:2401.11324* (2024).
- [72] Jon M Kleinberg. 2000. Navigation in a small world. *Nature* 406, 6798 (2000), 845–845.
- [73] Haridimos Kondylakis, Niv Dayan, Kostas Zoumpatianos, and Themis Palpanas. 2019. Coconut: sortable summarizations for scalable indexes over static and streaming data series. *The VLDB Journal* 28 (2019), 847–869.
- [74] Leonardo Kuffo and Peter Boncz. 2025. Bang for the Buck: Vector Search on Cloud CPUs. In *Proceedings of the 21st International Workshop on Data Management on New Hardware*. 1–8.
- [75] Leonardo Kuffo, Elena Krippner, and Peter Boncz. 2025. PDX: A Data Layout for Vector Similarity Search. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.
- [76] Yifan Lei, Qiang Huang, Mohan Kankanalli, and Anthony Tung. 2019. Sublinear time nearest neighbor search over generalized weighted space. In *International Conference on Machine Learning*. PMLR, 3773–3781.
- [77] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and others. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [78] Hui Li, Shiyuan Deng, Xiao Yan, Xiangyu Zhi, and James Cheng. 2025. SAQ: Pushing the Limits of Vector Quantization through Code Adjustment and Dimension Segmentation. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–25.
- [79] Huayang Li, Yixuan Su, Deng Cai, Yan Wang, and Lemao Liu. 2022. A survey on retrieval-augmented text generation. *arXiv preprint arXiv:2202.01110* (2022).
- [80] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2019. Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement. *IEEE Transactions on Knowledge and Data Engineering* 32, 8 (2019), 1475–1488.
- [81] Zhaoheng Li, Silu Huang, Wei Ding, Yongjoo Park, and Jianjun Chen. 2025. SIEVE: Effective Filtered Vector Search with Collection of Indexes. *arXiv preprint arXiv:2507.11907* (2025).
- [82] Anqi Liang, Pengcheng Zhang, Bin Yao, Zhongpu Chen, Yitong Song, and Guangxu Cheng. 2024. UNIFY: Unified Index for Range Filtered Approximate Nearest Neighbors Search. *arXiv preprint arXiv:2412.02448* (2024).
- [83] Michele Linardi and Themis Palpanas. 2020. Scalable data series subsequence matching with ULISSE. *The VLDB Journal* 29, 6 (2020), 1449–1474.
- [84] Qiuyu Liu, Yanlin Qi, Siyuan Han, Jingshu Peng, Jin Li, and Lei Chen. 2025. Not Small Enough? SegPQ: A Learned Approach to Compress Product Quantization Codebooks. *Proceedings of the VLDB Endowment* 18, 11 (2025), 3730–3743.
- [85] David G. Lowe. 2004. Distinctive Image Features from Scale-Invariant Keypoints. *International Journal of Computer Vision* 60 (2004), 91–110. <https://api.semanticscholar.org/CorpusID:174065>
- [86] Kejing Lu and Mineichi Kudo. 2020. R2LSH: A Nearest Neighbor Search Scheme Based on Two-dimensional Projected Spaces. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. 1045–1056. <https://doi.org/10.1109/ICDE48307.2020.00095>
- [87] J MacQueen. 1967. Multivariate observations. In *Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability*, Vol. 1. 281–297.
- [88] Vasilis Mageirakos, Bowen Wu, and Gustavo Alonso. 2025. Cracking Vector Search Indexes. *arXiv preprint arXiv:2503.01823* (2025).
- [89] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.
- [90] Jason Mohoney, Devesh Sarda, Mengze Tang, Shihabur Rahman Chowdhury, Anil Pacaci, Ihab F Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. 2025. Quake: Adaptive indexing for vector search. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 153–169.
- [91] Gonzalo Navarro. 2002. Searching in metric spaces by spatial approximation. *The VLDB Journal* 11 (2002), 28–46.
- [92] Peter Bjorn Nemenyi. 1963. *Distribution-free multiple comparisons*. Princeton University.
- [93] Aude Oliva and Antonio Torralba. 2001. Modeling the Shape of the Scene: A Holistic Representation of the Spatial Envelope. *International Journal of Computer Vision* 42 (2001), 145–175. <https://api.semanticscholar.org/CorpusID:11664336>
- [94] John Paparrizos, Ikdradya Edian, Chunwei Liu, Aaron J Elmore, and Michael J Franklin. 2022. Fast adaptive similarity search through variance-aware quantization. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2969–2983.
- [95] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. Acorn: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.
- [96] Loïc Paulevé, Hervé Jégou, and Laurent Amsaleg. 2010. Locality sensitive hashing: A comparison of hash function types and querying mechanisms. *Pattern recognition letters* 31, 11 (2010), 1348–1358.
- [97] Botoa Peng, Panagiota Fatourou, and Themis Palpanas. 2021. Fast data series indexing for in-memory data. *The VLDB Journal* 30, 6 (2021), 1041–1067.
- [98] Botoa Peng, Panagiota Fatourou, and Themis Palpanas. 2021. SING: Sequence indexing using GPUs. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 1883–1888.
- [99] Zhencan Peng, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2025. Dynamic Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 18, 10 (2025), 3256–3268.
- [100] Jeffrey Pennington, Richard Socher, and Christopher D Manning. 2014. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. 1532–1543.
- [101] Chanop Silpa-Anan and Richard Hartley. 2008. Optimised KD-trees for fast image descriptor matching. In *2008 IEEE conference on computer vision and*

- pattern recognition. IEEE, 1–8.
- [102] Soldi, S., Beckmann, V., Baumgartner, W. H., Ponti, G., Shrader, C. R., Lubiński, P., Krimm, H. A., Mattana, F., and Tueller, J. 2014. Long-term variability of AGN at hard X-rays. *A&A* 563 (2014), A57. <https://doi.org/10.1051/0004-6361/201322653>
- [103] Yifeng Song, Chenjie Liu, Rongrong Zhang, Danyang Zhu, and Zhongfeng Wang. 2025. An Efficient FPGA Implementation of Approximate Nearest Neighbor Search. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* (2025).
- [104] Philip Sun, David Simcha, Dave Dopson, Ruiqi Guo, and Sanjiv Kumar. 2023. SOAR: Improved Indexing for Approximate Nearest Neighbor Search. In *Neural Information Processing Systems*. <https://arxiv.org/abs/2404.00774>
- [105] Yuxi Sun, Yunming Ye, Jian Kang, Ruben Fernandez-Beltran, Xutao Li, Zhenyu Xiong, Xu Huang, and Antonio Plaza. 2023. Consistency center-based deep cross-modal hashing for multisource remote sensing image retrieval. *IEEE Transactions on Geoscience and Remote Sensing* 61 (2023), 1–16.
- [106] Mariano Tepper, Ishwar Singh Bhati, Cecilia Aguerrebere, and Ted Willke. 2024. GleanVec: Accelerating vector search with minimalist nonlinear dimensionality reduction. *arXiv preprint arXiv:2410.22347* (2024).
- [107] Bing Tian, Haikun Liu, Yuhang Tang, Shihai Xiao, Zhuohui Duan, Xiaofei Liao, Xuechang Zhang, Junhua Zhu, and Yu Zhang. 2024. Fusionnans: An efficient cpu/gpu cooperative processing architecture for billion-scale approximate nearest neighbor search. *arXiv preprint arXiv:2409.16576* (2024).
- [108] Yao Tian, Xi Zhao, and Xiaofang Zhou. 2023. DB-LSH 2.0: Locality-sensitive hashing with query-based dynamic bucketing. *IEEE Transactions on Knowledge and Data Engineering* 36, 3 (2023), 1000–1015.
- [109] Nitish Upreti, Harsha Vardhan Simhadri, Hari Sudan Sundar, Krishnan Sundaram, Samer Boshra, Balachandrar Perumalswamy, Shivam Atri, Martin Chisholm, Revti Raman Singh, Greg Yang, and others. 2025. Cost-Effective, Low Latency Vector Search with Azure Cosmos DB. *arXiv preprint arXiv:2505.05885* (2025).
- [110] Sairaj Voruganti and M. Tamer Özsu. 2025. MIRAGE-ANNS: Mixed Approach Graph-based Indexing for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 3 (June 2025). <https://doi.org/10.1145/3725325> Place: New York, NY, USA.
- [111] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, and others. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data*. 2614–2627.
- [112] Mengzhao Wang, Lingwei Lv, Xiaoliang Xu, Yuxiang Wang, Qiang Yue, and Jiongkang Ni. 2022. Navigable proximity graph-driven native hybrid queries with structured and unstructured constraints. *arXiv preprint arXiv:2203.13601* (2022).
- [113] Qitong Wang, Ioana Ileana, and Themis Palpanas. 2025. Leafi: Data Series Indexes on Steroids with Learned Filters. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–27.
- [114] Runhui Wang and Dong Deng. 2020. DeltaPQ: lossless product quantization code compression for high dimensional similarity search. *Proceedings of the VLDB Endowment* 13, 13 (2020), 3603–3616.
- [115] Wenping Wang, Yunxi Guo, Chiyao Shen, Shuai Ding, Guangdeng Liao, Hao Fu, and Pramodh Karanth Prabhakar. 2023. Integrity and junkiness failure handling for embedding-based retrieval: A case study in social network search. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 3250–3254.
- [116] Yang Wang, Peng Wang, Jian Pei, Wei Wang, and Sheng Huang. 2013. A data-adaptive and dynamic segmentation index for whole matching on time series. *Proceedings of the VLDB Endowment* 6, 10 (2013), 793–804.
- [117] Zihao Wang. 2025. Graph Library for Approximate Similarity Search. <https://github.com/zilliztech/pyglass>
- [118] Zeyu Wang, Peng Wang, Themis Palpanas, and Wei Wang. 2023. Graph-and Tree-based Indexes for High-dimensional Vector Similarity Search: Analyses, Comparisons, and Future Directions. *IEEE Data Eng. Bull.* 46, 3 (2023), 3–21.
- [119] Zeyu Wang, Qitong Wang, Peng Wang, Themis Palpanas, and Wei Wang. 2023. Dumpy: A compact and adaptive index for large data series collections. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27.
- [120] Roger Weber, Hans-Jörg Schek, and Stephen Blott. 1998. A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces. In *VLDB*, Vol. 98. 194–205.
- [121] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. Analyticdcb-v: A hybrid analytical engine towards query fusion for structured and unstructured data. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3152–3165.
- [122] Dongtao Wei, Kaixiang Zhuang, Lei Ai, Qunlin Chen, Wenjing Yang, Wei Liu, Kangcheng Wang, Jiangzhou Sun, and Jiang Qiu. 2018. Structural and functional brain scans from the cross-sectional Southwest University adult lifespan dataset. *Scientific Data* 5, 1 (July 2018), 180134. <https://doi.org/10.1038/sdata.2018.134>
- [123] Jiuqi Wei, Xiaodong Lee, Zhenyu Liao, Themis Palpanas, and Botao Peng. 2025. Subspace collision: an efficient and accurate framework for high-dimensional approximate nearest neighbor search. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–29.
- [124] Jiuqi Wei, Botao Peng, Xiaodong Lee, and Themis Palpanas. 2024. Det-lsh: a locality-sensitive hashing scheme with dynamic encoding tree for approximate nearest neighbor search. *arXiv preprint arXiv:2406.10938* (2024).
- [125] Frank Wilcoxon. 1992. Individual comparisons by ranking methods. In *Breakthroughs in statistics: Methodology and distribution*. Springer, 196–202.
- [126] Jack Woollam, Jannes Münchmeyer, Frederik Tilmann, Andreas Rietbrock, Dietrich Lange, Thomas Bornstein, Tobias Diehl, Carlo Giunchi, Florian Haslinger, Dario Jozinović, Alberto Michelini, Joachim Saul, and Hugo Soto. 2022. SeisBench—A Toolbox for Machine Learning in Seismology. *Seismological Research Letters* 93, 3 (March 2022), 1695–1709. <https://doi.org/10.1785/0220210324> eprint: <https://pubs.geoscienceworld.org/ssa/srl/article-pdf/93/3/1695/5596371/srl-2021324.1.pdf>.
- [127] Wei Wu, Junlin He, Yu Qiao, Guoheng Fu, Li Liu, and Jin Yu. 2022. HQANN: Efficient and robust similarity search for hybrid queries with structured and unstructured constraints. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 4580–4584.
- [128] Jingyi Xi, Chenghao Mo, Ben Karsin, Artem Chirkin, Mingqin Li, and Minjia Zhang. 2025. VecFlow: A High-Performance Vector Data Management System for Filtered-Search on GPUs. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–27.
- [129] Qian Xu, Feng Zhang, Chengxi Li, Lei Cao, Zheng Chen, Jidong Zhai, and Xiaoyong Du. 2025. Harmony: A scalable distributed vector database for high-throughput approximate nearest neighbor search. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–28.
- [130] Yuxuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S Jensen. 2024. irangraph: Improvising range-dedicated graphs for range-filtering nearest neighbor search. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–26.
- [131] Djamel-Edine Yagoubi, Reza Akbarinia, Florent Masseglia, and Themis Palpanas. 2018. Massively distributed time series indexing and querying. *IEEE Transactions on Knowledge and Data Engineering* 32, 1 (2018), 108–120.
- [132] Mingyu Yang, Jiabao Jin, Xiangyu Wang, Zhitao Shen, Wei Jia, Wentao Li, and Wei Wang. 2024. Bridging Speed and Accuracy to Approximate-Nearest Neighbor Search. *CoRR* (2024).
- [133] Mingyu Yang, Wentao Li, Jiabao Jin, Xiaoyao Zhong, Xiangyu Wang, Zhitao Shen, Wei Jia, and Wei Wang. 2024. Effective and General Distance Computation for Approximate Nearest Neighbor Search. (2024). <https://arxiv.org/abs/2404.16322>
- [134] Shuo Yang, Jiadong Xie, Yingfan Liu, Jeffrey Xu Yu, Xiyue Gao, Qianru Wang, Yanguo Peng, and Jiangtao Cui. 2024. Revisiting the index construction of proximity graph-based approximate nearest neighbor search. *arXiv preprint arXiv:2410.01231* (2024).
- [135] Wen Yang, Tao Li, Gai Fang, and Hong Wei. 2020. Pase: Postgresql ultra-high-dimensional approximate nearest neighbor search extension. In *Proceedings of the 2020 ACM SIGMOD international conference on management of data*. 2241–2253.
- [136] Peter N Yianilos. 1993. Data structures and algorithms for nearest neighbor search in general metric spaces. In *Soda*, Vol. 93. 311–21.
- [137] Ziqi Yin, Jianyang Gao, Pasquale Balsebre, Gao Cong, and Cheng Long. 2025. DEG: Efficient Hybrid Vector Search Using the Dynamic Edge Navigation Graph. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–28.
- [138] Kai Yu, Rao Ma, Kaiyu Shi, and Qi Liu. 2020. Neural network language model compression with product quantization and soft binarization. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 28 (2020), 2438–2449.
- [139] Wei Yuan and Xi Jin. 2025. FANNS: An FPGA-Based Approximate Nearest-Neighbor Search Accelerator. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* (2025).
- [140] Chao Zhang and Renée J Miller. 2025. Distribution-Aware Exploration for Adaptive HNSW Search. *arXiv preprint arXiv:2512.06636* (2025).
- [141] Fangyuan Zhang, Mengxu Jiang, Guanhao Hou, Jieming Shi, Hua Fan, Wencho Zhou, Feifei Li, and Sibao Wang. 2025. Efficient Dynamic Indexing for Range Filtered Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.
- [142] Hailin Zhang, Xiaodong Ji, Yilin Chen, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, Weipeng Chen, and Bin Cui. 2025. Pqcache: Product quantization-based kvcache for long context llm inference. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–30.
- [143] Haokui Zhang, Buzhou Tang, Wenze Hu, and Xiaoyu Wang. 2022. Connecting kspace spaces with transformer for approximate nearest neighbor search. In *European Conference on Computer Vision*. Springer, 515–530.
- [144] Ting Zhang, Chao Du, and Jingdong Wang. 2014. Composite quantization for approximate nearest neighbor search. In *International Conference on Machine Learning*. PMLR, 838–846.
- [145] Weijie Zhao, Shulong Tan, and Ping Li. 2020. Song: Approximate nearest neighbor search on gpu. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 1033–1044.

- [146] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. 2023. Towards efficient index construction and approximate nearest neighbor search in high-dimensional spaces. *Proceedings of the VLDB Endowment* 16, 8 (2023), 1979–1991.
- [147] Bolong Zheng, Zhao Xi, Lianggui Weng, Nguyen Quoc Viet Hung, Hang Liu, and Christian S Jensen. 2020. PM-LSH: A fast and accurate LSH framework for high-dimensional approximate NN search. *Proceedings of the VLDB Endowment* 13, 5 (2020), 643–655.
- [148] Xiaoyao Zhong, Haotian Li, Jiabao Jin, Mingyu Yang, Deming Chu, Xiangyu Wang, Zhitao Shen, Wei Jia, George Gu, Yi Xie, and others. 2025. VSAG: An Optimized Search Framework for Graph-based Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2503.17911* (2025).
- [149] Wengang Zhou, Yijuan Lu, Houqiang Li, and Qi Tian. 2012. Scalar quantization for large scale image search. In *Proceedings of the 20th ACM international conference on Multimedia*. 169–178.
- [150] Kostas Zoumpatianos, Stratos Idreos, and Themis Palpanas. 2016. ADS: the adaptive data series index. *The VLDB Journal* 25 (2016), 843–866.